

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_5070b515-30d\agent-a060262c33ba5d03e.jsonl`  
- SHA-256: `1e2e08a3d6541e8f1d1c4ff4d0a092c9d2b6a52dba48c94dc3ebc26b7882d6fe`  
- Source modified: `2026-05-31T14:30:03+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_5070b515-30d`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T14:27:51.942Z)

Project context - muneem: a SINGLE-USER, LOCAL-FIRST personal-finance bookkeeper in Python (3.11-3.13), developed on Windows, must also run on Linux. It parses bank/credit-card/investment statement PDFs into a LOCAL SQLite ledger (stdlib sqlite3 today; schema v4: documents/offers/per-bank txn tables). The user now wants to store REAL financial numbers and run BASIC SQL QUERIES (SELECT/WHERE/aggregations) over them, and wants the database "hardened" (encryption-at-rest).

HARD CONSTRAINTS (a tool that violates these is unusable):

1. 100% LOCAL. No cloud KMS/SaaS. Key stays on the user machine (OS keychain via the "keyring" lib is available).
2. Usable from Python, ideally close to the stdlib sqlite3 API. Must KEEP SQL QUERYABILITY (WHERE/ORDER BY/aggregates over stored numbers) - so TRANSPARENT full-DB page encryption is strongly preferred over per-field app-level encryption that breaks queries.
3. License MUST be permissive & commercially safe: MIT / BSD / Apache-2.0 / ISC / HPND - for BOTH the Python binding AND every native dependency. AVOID/FLAG anything GPL/LGPL/AGPL. (PyMuPDF AGPL is already banned here.) OpenSSL is acceptable.
4. Cross-platform prebuilt WHEELS (Windows + Linux, cp311-cp313) strongly preferred - avoid a from-source C/JVM build burden.

Cite authoritative sources (project LICENSE files, PyPI pages, SPDX, official docs) with URLs.

ANGLE: Local-first finance app practice + operational realities.

Survey how real LOCAL-FIRST / self-hosted personal-finance tools handle at-rest encryption, and the operational realities. (a) What do Actual Budget, Firefly III, GnuCash, beancount / ledger-cli, Maybe, and similar do about encrypting the local DB at rest (default plaintext? optional encryption? rely on FDE?) - cite docs. (b) Operational realities of adopting SQLCipher/SQLite3MC: backup/restore of an encrypted DB, migrations, debugging (cannot open with plain sqlite3 CLI), perf on typical sizes, corruption risk, WAL. (c) Notable security advisories/CVEs for SQLCipher or SQLite3MC. (d) A pragmatic PHASED recommendation: is encryption-at-rest worth adopting NOW for a tool just starting to store real numbers, or is FDE+keychain+tight file perms enough for the current phase with transparent encryption added at a defined trigger? Cite URLs.

2. assistant (2026-05-31T14:27:55.005Z)

I'll help you research encryption options for your local-first personal finance app. Let me start by searching the muneem codebase to understand the current state, then conduct a

comprehensive survey of how other tools handle this.

3. user (2026-05-31T14:27:55.839Z)

```
.venv\Lib\site-packages\pip\__init__.py
.venv\Lib\site-packages\pip\__main__.py
.venv\Lib\site-packages\pip\_pip-runner__.py
.venv\Lib\site-packages\pip\_internal\__init__.py
.venv\Lib\site-packages\pip\_internal\build_env.py
.venv\Lib\site-packages\pip\_internal\cache.py
.venv\Lib\site-packages\pip\_internal\configuration.py
.venv\Lib\site-packages\pip\_internal\exceptions.py
.venv\Lib\site-packages\pip\_internal\main.py
.venv\Lib\site-packages\pip\_internal\pyproject.py
.venv\Lib\site-packages\pip\_internal\self_outdated_check.py
.venv\Lib\site-packages\pip\_internal\cli\__init__.py
.venv\Lib\site-packages\pip\_internal\wheel_builder.py
.venv\Lib\site-packages\pip\_internal\cli\autocompletion.py
.venv\Lib\site-packages\pip\_internal\cli\base_command.py
.venv\Lib\site-packages\pip\_internal\cli\cmdoptions.py
.venv\Lib\site-packages\pip\_internal\cli\command_context.py
.venv\Lib\site-packages\pip\_internal\cli\index_command.py
.venv\Lib\site-packages\pip\_internal\cli\main.py
.venv\Lib\site-packages\pip\_internal\cli\main_parser.py
.venv\Lib\site-packages\pip\_internal\cli\parser.py
.venv\Lib\site-packages\pip\_internal\cli\progressBars.py
.venv\Lib\site-packages\pip\_internal\cli\req_command.py
.venv\Lib\site-packages\pip\_internal\cli\spinners.py
.venv\Lib\site-packages\pip\_internal\cli\status_codes.py
.venv\Lib\site-packages\pip\_internal\commands\cache.py
.venv\Lib\site-packages\pip\_internal\commands\__init__.py
.venv\Lib\site-packages\pip\_internal\commands\check.py
.venv\Lib\site-packages\pip\_internal\commands\completion.py
.venv\Lib\site-packages\pip\_internal\commands\configuration.py
.venv\Lib\site-packages\pip\_internal\commands\debug.py
.venv\Lib\site-packages\pip\_internal\commands\download.py
.venv\Lib\site-packages\pip\_internal\commands\freeze.py
.venv\Lib\site-packages\pip\_internal\commands\hash.py
.venv\Lib\site-packages\pip\_internal\commands\help.py
.venv\Lib\site-packages\pip\_internal\commands\index.py
.venv\Lib\site-packages\pip\_internal\commands\inspect.py
.venv\Lib\site-packages\pip\_internal\commands\install.py
.venv\Lib\site-packages\pip\_internal\commands\list.py
.venv\Lib\site-packages\pip\_internal\commands\lock.py
.venv\Lib\site-packages\pip\_internal\commands\search.py
.venv\Lib\site-packages\pip\_internal\commands\show.py
.venv\Lib\site-packages\pip\_internal\commands\uninstall.py
.venv\Lib\site-packages\pip\_internal\commands\wheel.py
.venv\Lib\site-packages\pip\_internal\distributions\__init__.py
.venv\Lib\site-packages\pip\_internal\distributions\base.py
.venv\Lib\site-packages\pip\_internal\distributions\installed.py
.venv\Lib\site-packages\pip\_internal\distributions\sdist.py
.venv\Lib\site-packages\pip\_internal\distributions\wheel.py
.venv\Lib\site-packages\pip\_internal\index\__init__.py
.venv\Lib\site-packages\pip\_internal\index\collector.py
.venv\Lib\site-packages\pip\_internal\index\package_finder.py
.venv\Lib\site-packages\pip\_internal\index\sources.py
.venv\Lib\site-packages\pip\_internal\locations\__init__.py
.venv\Lib\site-packages\pip\_internal\locations\_distutils.py
.venv\Lib\site-packages\pip\_internal\locations\_sysconfig.py
.venv\Lib\site-packages\pip\_internal\locations\base.py
.venv\Lib\site-packages\pip\_internal\metadata\_json.py
.venv\Lib\site-packages\pip\_internal\metadata\__init__.py
.venv\Lib\site-packages\pip\_internal\metadata\base.py
.venv\Lib\site-packages\pip\_internal\metadata\pkg_resources.py
```

```
.venv\Lib\site-packages\pip\_internal\metadata\importlib\__init__.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\_compat.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\_dists.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\_envs.py
.venv\Lib\site-packages\pip\_internal\models\__init__.py
.venv\Lib\site-packages\pip\_internal\models\candidate.py
.venv\Lib\site-packages\pip\_internal\models\direct_url.py
.venv\Lib\site-packages\pip\_internal\models\format_control.py
.venv\Lib\site-packages\pip\_internal\models\index.py
.venv\Lib\site-packages\pip\_internal\models\installation_report.py
.venv\Lib\site-packages\pip\_internal\models\link.py
.venv\Lib\site-packages\pip\_internal\models\release_control.py
.venv\Lib\site-packages\pip\_internal\models\scheme.py
.venv\Lib\site-packages\pip\_internal\models\search_scope.py
.venv\Lib\site-packages\pip\_internal\models\selection_prefs.py
.venv\Lib\site-packages\pip\_internal\models\target_python.py
.venv\Lib\site-packages\pip\_internal\models\wheel.py
.venv\Lib\site-packages\pip\_internal\network\auth.py
.venv\Lib\site-packages\pip\_internal\network\__init__.py
.venv\Lib\site-packages\pip\_internal\network\cache.py
.venv\Lib\site-packages\pip\_internal\network\download.py
.venv\Lib\site-packages\pip\_internal\network\lazy_wheel.py
.venv\Lib\site-packages\pip\_internal\network\session.py
.venv\Lib\site-packages\pip\_internal\network\utils.py
.venv\Lib\site-packages\pip\_internal\network\xmlrpc.py
.venv\Lib\site-packages\pip\_internal\operations\check.py
.venv\Lib\site-packages\pip\_internal\operations\__init__.py
.venv\Lib\site-packages\pip\_internal\operations\freeze.py
.venv\Lib\site-packages\pip\_internal\operations\prepare.py
.venv\Lib\site-packages\pip\_internal\operations\build\build_tracker.py
.venv\Lib\site-packages\pip\_internal\operations\build\__init__.py
.venv\Lib\site-packages\pip\_internal\operations\build\metadata.py
.venv\Lib\site-packages\pip\_internal\operations\build\metadata_editable.py
.venv\Lib\site-packages\pip\_internal\operations\build\wheel.py
.venv\Lib\site-packages\pip\_internal\operations\build\wheel_editable.py
.venv\Lib\site-packages\pip\_internal\operations\install\__init__.py
.venv\Lib\site-packages\pip\_internal\operations\install\wheel.py
.venv\Lib\site-packages\pip\_internal\req\constructors.py
.venv\Lib\site-packages\pip\_internal\req\__init__.py
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-05-31T14:27:56.174Z)

```
CLAUDE.md
testdata\promo-emails\README.md
docs\TESTING.md
.venv\Lib\site-packages\pip\_vendor\idna\LICENSE.md
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip\_vendor\idna\LICENSE.md
.venv\Lib\site-packages\typer\.agents\skills\typer\SKILL.md
.pytest_cache\README.md
docs\prior-art-email-mining.md
.venv\Lib\site-packages\pypdfium2-5.8.0.dist-
info\licenses\data\windows_x64\BUILD_LICENSES\libjpeg_turbo.md
.venv\Lib\site-packages\pyparsing\ai\best_practices.md
.venv\Lib\site-packages\idna-3.17.dist-info\licenses\LICENSE.md
DESIGN.md
docs\DEPENDENCIES.md
docs\ingesting-statements.md
README.md
docs\parser-architecture.md
docs\CURRENT_STATE.md
docs\onboarding-a-bank.md
docs\ROADMAP.md
```

5. user (2026-05-31T14:27:56.459Z)

```

9c13e69 feat(cli): `muneem eval` seed?golden onboarding harness + playbook (#33)
02cc372 feat(parsers): balance-delta reconstruction recovers Axis June 2025 (5/7 -> 6/7) (#32)
71869a0 test(ci): Layer-1 regression ? synthetic full-pipeline fixtures (HDFC + savings) (#31)
0d907f8 docs: capture L3 (DeepSeek-OCR) + golden-eval decisions; track Regression CI as a
critical item (#30)
f784e71 docs: track the regression-CI plan (synthetic cloud + self-hosted real) ? decided,
deferred (#29)
d4f071e feat(cli): wire hybrid password-unlock into `muneem ingest` (C.1) + setup doc (#28)
8b65ad4 docs: sync ROADMAP / CURRENT_STATE / parser-architecture to the shipped state (#27)
5f41467 feat(parsers): AU Bank parser + extract shared engine-driven savings path (#26)
5156b5a feat(parsers): Axis savings rewrite onto the concept core (clustering + total reconcile)
(#25)
2a589ee chore(refocus): remove ICICI; reframe docs to the whole-finance view (HDFC/Axis/AU)
(#24)
7ea7f7a feat(parsers): generic concept-mapping engine (L0/L1/L2) + profiles; ICICI runs through
it (#23)
7555fb2 feat(parsers): concept-core verifier + schema; HDFC refactored onto it (no behavior
change) (#22)
6f6141f docs: parser architecture proposal (concept-core, verification-first) [DRAFT] (#21)
128fc36 docs(roadmap): flag Axis savings parser as a must-fix (needs thorough rewrite) (#20)
8e1e2a8 fix(parsers): Axis savings refuses to store unreconciled rows (stop-gap) (#19)
bdee68d docs: CURRENT_STATE reflects Axis/ICICI now reconcile (shared validation) (#18)
90579f8 feat(parsers): retrofit ICICI with parse/persist split + reconciliation (#17)
1e12e21 feat(parsers): retrofit Axis with parse/persist split + reconciliation (#16)
988a3ca refactor(parsers): extract shared running-balance reconciler; HDFC adopts it (#15)
a40eebf feat(parsers): HDFC savings-statement parser with reconciliation + SUMMARY bookend (#14)

```

6. assistant (2026-05-31T14:28:00.313Z)

Now let me look at the project structure and documentation more closely:

7. user (2026-05-31T14:28:00.797Z)

```

1      # muneem
2
3      **Your automated muneem ? a personal-finance bookkeeper that pulls financial documents
from your inbox, unlocks them, and builds a local consolidated ledger.**
4
5      In traditional Indian households a *muneem* (????) was the trusted bookkeeper who
tracked every statement, bill, and transaction on the family's behalf. This project is the same
idea, automated and personal.
6
7      **Scope is the whole financial picture, not just banks.** Bank and credit-card
statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are core
targets**, parsed into the same normalized local ledger (with transaction/instrument
classification to follow). The parser core is deliberately *family-agnostic* ? adding an
instrument family is new data + a reconciliation rule, not a new pipeline.
8
9      ## What it does (intended)
10
11     Reads Gmail and Outlook for financial documents (bank statements, credit-card bills,
brokerage statements, mutual-fund reports, utility bills, grocery and shopping receipts),
decrypts the password-protected ones, parses them into a normalized local database, and surfaces
insights:
12
13     - Grocery/shopping price-tracking across merchants over time
14     - Investment portfolio timeline and cross-broker view
15     - Bank/card performance comparison
16     - Catalog of purchased items with their bills (warranty, returns, age)
17     - Forgotten deals and coupons sitting unread in your inbox
18
19     ## Status
20
21     **Early implementation.** The design and plan are settled ? see
[ `DESIGN.md`](./DESIGN.md) (architecture + threat model), [ `docs/ROADMAP.md`](./docs/ROADMAP.md)

```

```

(milestones), and [`docs/TESTING.md`](./docs/TESTING.md) (test strategy). The Python project
skeleton, CLI, and offline CI are in place; build proceeds by the roadmap's milestones. Prior
art: [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md).
22
23     **Ingesting bank statements** (HDFC / Axis / AU Bank, with automatic password unlock):
[`docs/ingesting-statements.md`](./docs/ingesting-statements.md).
24
25     ## Getting started
26
27     Runs on Linux, macOS, and Windows. Requires **Python ? 3.11** and git.
28
29     ### Linux / macOS
30
31     ```bash
32     python3 -m venv .venv
33     source .venv/bin/activate
34     pip install -e ".[dev]"
35
36     muneem --version
37     muneem info
38     muneem ingest testdata/promo-emails/dataquest.pdf    # -> JS231sz
39     muneem offers list
40     ```
41
42     Or use the `Makefile`: `make install`, `make test`, `make lint`, `make run`.
43
44     ### Windows (PowerShell)
45
46     ```powershell
47     python -m venv .venv
48     .venv\Scripts\Activate.ps1
49     pip install -e ".[dev]"
50     muneem info
51     ```
52
53     ### Optional: OCR for codes inside banner images
54
55     The `--ocr` flag uses **Tesseract** (Apache-2.0, fully local ? no paid API):
56
57     | OS | Install |
58     |---|---|
59     | Debian / Ubuntu | `sudo apt-get install tesseract-ocr` |
60     | Fedora | `sudo dnf install tesseract` |
61     | macOS | `brew install tesseract` |
62     | Windows | `winget install UB-Mannheim.TesseractOCR` |
63
64     muneem finds it on `PATH`, at the usual install locations, or via the
`MUNEEM_TESSERACT_CMD` env var. Then `muneem ingest <pdf> --ocr` (run OCR tests with `make test-ocr`).
65
66     ### Notes
67
68     - **Headless Linux:** secret storage (used once Gmail/unlock land) needs a keyring
backend ? a desktop keyring (GNOME Keyring / KWallet), or `pip install keyrings.alt` for a file
backend. Not needed yet.
69     - CI runs the offline suite on Linux across Python 3.11?3.13 on every push.
70
71     ## Privacy
72
73     Single-user, fully local by design. All raw statement data stays on the machine. See
`DESIGN.md` § Security for the threat model and standing rules.
74

```

```
1      # muneem ? Design Document
2
3      > Status: **draft, pre-implementation**. This document captures the vision, the pipeline
shape, the security posture, and the open decisions to resolve before code lands. It is
intentionally opinionated where the author has a strong view, and explicitly undecided where the
trade-offs need a human call.
4
5      ---
6
7      ## 1. What muneem is
8
9      **muneem** is a personal-finance bookkeeper that ingests financial documents from the
user's email accounts, unlocks the password-protected ones, parses them into a normalized
schema, stores them in a local database, and surfaces analytics across the user's full financial
life.
10
11     The name is the Hindi word for the traditional household/business bookkeeper ? a trusted
figure who maintained the family ledger by hand. This project is the same role, automated.
12
13     **Single user. Local-first. Privacy-critical.**
14
15     ## 2. Why
16
17     The user's financial life today is fragmented across many silos:
18
19     - 3?5 bank accounts (statements arrive monthly as password-protected PDFs)
20     - Multiple credit cards (monthly bills, also password-protected PDFs)
21     - Brokerage and mutual-fund accounts (quarterly statements, portfolio snapshots)
22     - Grocery and shopping receipts (Swiggy, Zomato, Amazon, Flipkart, BigBasket, Blinkit ?
mostly plain HTML/PDF in email body)
23     - Utility bills, insurance premiums, EMI schedules
24     - Promotional offers and coupons (often forgotten)
25
26     Each silo has its own portal, login, and format. There is no consolidated view. **The
inbox is the only place where they already converge.**
27
28     Concrete use-cases the user wants once this exists:
29
30     1. **Grocery & shopping price intelligence** ? track what was bought, where, and at what
price, over time. Detect when a merchant is no longer competitive on staples.
31     2. **Investment portfolio timeline** ? a single time-series of holdings across brokers,
so allocation drift and rebalancing decisions are obvious.
32     3. **Bank/card comparison** ? fees paid, interest earned, reward redemption efficiency,
surfaced as objective numbers.
33     4. **Item catalog** ? every significant purchase linked to its receipt/invoice, so
warranty windows, return windows, and replacement cycles are queryable.
34     5. **Forgotten-deals reminder** ? surface the unused coupons and offers already sitting
in the inbox (this is the original goal of the predecessor project; see § 11).
35
36     ## 3. Origin
37
38     muneem is the successor to `~/Projects/Email-Mining`, a small prototype the user built
earlier that extracted promo codes from manually-saved email PDFs using PyMuPDF + Gemini for
image OCR. Same insight ("the inbox contains structured value I'm forgetting"), generalized
scope.
39
40     See [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md) for what it
did, what to inherit, and what not to.
41
42     ## 4. The pipeline (five stages)
43
44     Each stage is independent enough to develop and test in isolation. They communicate
through well-defined data boundaries (raw files on disk ? unlocked files on disk ? structured
records in DB ? derived views).
45
```

```

46     ### Stage 1 ? Email ingestion
47
48     **Goal:** pull messages and attachments from Gmail and Outlook into a local staging
49     area.
50     - **Gmail:** [Gmail API](https://developers.google.com/gmail/api) over OAuth2. Read-only
51     scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf
52     from:hdfc newer_than:90d`) which is far cheaper than pulling everything and filtering locally.
53     Has good rate limits for personal use.
54     - **Outlook:** [Microsoft Graph API](https://learn.microsoft.com/en-
55     us/graph/api/resources/mail-api-overview) over OAuth2. Read-only scope (`Mail.Read`). Similar
56     search capability.
57     - **Why not IMAP:** worse search, no labels, harder OAuth, and we lose Gmail's powerful
58     query syntax. Use the native APIs.
59
60     Mechanically straightforward. The main design questions are: **how is "financial
61     document" detected** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and
62     **how is incremental sync state tracked** (Gmail `historyId` / Graph `deltaLink`).
63
64     ### Stage 2 ? Attachment unlocking
65
66     **Goal:** decrypt password-protected PDFs.
67
68     The decryption itself is trivial ? every mainstream PDF library handles the standard
69     security handler. In Go, `github.com/pdfcpu/pdfcpu`; in Python, `pikepdf` or `pypdf`.
70
71     The real design question is **where the passwords come from.** Indian bank and CC
72     statement passwords are almost always derived from user attributes:
73
74     - "First 4 letters of name (uppercase) + DDMM of DOB" ? common at HDFC, Axis
75     - "PAN number" ? common at some brokers
76     - "Last 4 digits of card number" ? common for CC statements
77     - "Customer ID" ? some banks
78     - "Mobile number" ? utilities
79
80     Three plausible approaches, each with trade-offs:
81
82     | Approach | Pro | Con |
83     |---|---|---|
84     | **Per-sender rule template** the user configures once (e.g., `from:alerts@hdfcbank.net
85     ? first4(name)+ddmm(dob)`) | Deterministic, no guessing, auditable | Requires upfront setup;
86     rules must be maintained as banks change formats |
87     | **Candidate dictionary** (try a small list of derived strings per PDF) | Zero config;
88     "just works" | Brute-forceable feel; fails noisily on unknown senders |
89     | **Interactive prompt** (ask the user the first time, cache for that sender) | Safe,
90     simple | Friction during initial backfill of years of statements |
91
92     A hybrid is likely best: candidate dictionary first, fall back to rule template, fall
93     back to interactive prompt. **Decision parked.** See § 8.
94
95     ### Stage 3 ? Document parsing
96
97     **This is the hard 80% of the project.** Every issuer's PDF layout is different and
98     changes over time.
99
100    The space of approaches, in increasing robustness and cost:
101
102    1. **Per-issuer regex/template parsers.** Precise; brittle. One parser per format,
103    breaks when the issuer redesigns. Realistic for the top ~5 senders that dominate volume.
104    2. **Text-layer extraction + structural heuristics.** Pull text with `pdfminer.six` /
105    `pdfplumber` (both MIT); rely on coordinates and font runs to recover tables. Works well for
106    "born digital" PDFs (most bank statements); fails on scanned/image PDFs. (We avoid
107    PyMuPDF/`fitz` ? it is AGPL; see § 5 rule 8.)
108    3. **OCR (Tesseract) for image-based PDFs and embedded raster content.** Slower; need
109    layout reconstruction.

```

89 4. ****LLM-assisted structured extraction.**** Hand page text (+ optionally page rasters) to a model with a target JSON schema. Generalizes across layouts; expensive per page; ****must be local for sensitive document categories****. Options: Ollama-hosted models (Llama 3, Qwen, Mistral), or a small purpose-built model.

90

91 In practice this will be ****a hybrid****: deterministic template parsers for the top ~5 high-volume senders (where they pay for themselves), LLM-assisted extraction as the long-tail fallback. Promo emails and other non-sensitive categories can use cloud LLMs; bank/card/investment statements ****must not**** without explicit per-category approval.

92

93 Embedded image deduplication (Email-Mining used MD5 hash + SSIM) is a real win here: bank statements repeat the same logo/letterhead on every page, and we don't want to OCR that 30 times per document.

94

95 **### Stage 4 ? Normalization & storage**

96

97 ****Goal:**** map heterogeneous parser outputs into one schema, persist locally.

98

99 Storage: ****SQLite****. Single file, no server, easy to back up, easy to encrypt at rest (SQLCipher), good enough performance for years of personal data. The schema needs to span:

100

101 - `documents` ? every ingested file with sender, date, type, hash, path

102 - `accounts` ? bank accounts, cards, broker accounts, identified by issuer + last-4 or similar

103 - `transactions` ? line items from statements, normalized (date, amount, currency, counterparty, category, source_document)

104 - `holdings` ? point-in-time snapshots of investment positions per account

105 - `merchants` ? canonicalized merchant identities (so "SWIGGY*BANGALORE", "Swiggy Instamart", and "SWIGGY BNGLR" collapse to one)

106 - `items` ? line-items from itemized receipts (grocery, shopping)

107 - `offers` ? promotional offers/coupons with expiry, source, redemption status

108

109 The merchant canonicalization step is non-trivial ? it deserves its own thought (fuzzy match + manual override table is the usual answer).

110

111 **### Stage 5 ? Analytics & insights**

112

113 ****Goal:**** turn the normalized DB into the use-cases in § 2.

114

115 Once the data is in the schema, the analytics are mostly SQL + light application logic. Likely surface: a local CLI initially, possibly a small local web UI later. Notebook-style exploration is fine for prototyping queries before they're promoted to first-class views.

116

117 ---

118

119 **## 5. Security posture (first class)**

120

121 The threat model deserves to be stated plainly: ****muneem aggregates the user's entire financial life ? every account, every transaction, every holding ? into one local store. That store is a honeypot.**** A leak would be catastrophic.

122

123 Standing rules:

124

125 1. ****All raw statement data stays on the local machine.**** No cloud sync of the staging dir or the DB.

126 2. ****No cloud LLM APIs for sensitive document categories**** (bank, CC, broker, MF, insurance) without explicit per-category user approval discussed in the current session. The default extraction path for these is local (Tesseract + local LLM via Ollama, or deterministic parsers).

127 3. ****Non-sensitive categories**** (promo emails, generic newsletters, public receipts) may use cloud LLMs after a privacy review of the specific content type.

128 4. ****DB encrypted at rest**** (SQLCipher or equivalent), key never in the repo.

129 5. ****OAuth tokens**** stored in the OS keychain (Windows Credential Manager / macOS Keychain), not on disk.

130 6. ****Logs must not contain extracted statement text.**** Sanitize aggressively; redact

account numbers, balances, line-items.

131 7. **Nothing sensitive ever in git.** ``.gitignore`` blocks real PDFs, the DB file, ``.env``, and password files. Do not weaken it.

132 8. **Commercial-license safety.** Only dependencies under permissive, commercial-safe licenses (MIT, BSD, Apache-2.0, ISC, HPND). No copyleft (GPL/LGPL/**AGPL**) or source-available licenses. **PyMuPDF/`fitz`** is AGPL-3.0 and is banned **? use pdfminer.six (MIT) / pypdfium2 (Apache/BSD)** instead. Enforced by ``tests/test_dependency_licenses.py``; tracked in `[`docs/DEPENDENCIES.md`](./docs/DEPENDENCIES.md)`.

133 9. **Minimize paid dependencies.** Prefer local / self-hosted / open-source (and, where needed, own-trained) models and tools. Paid APIs **? cloud LLMs, paid OCR/vision ? are a **fallback only**, gated behind explicit per-use approval. This extends rule 2 from privacy to cost.**

134

135 **## 6. Non-goals (for v1)**

136

137 - **Multi-user / SaaS / hosted product.** Single user, single machine.

138 - **Cloud sync of raw data.** Period.

139 - **Cracking unknown passwords.** muneem decrypts PDFs the user is authorized to read, using passwords the user knows or can derive.

140 - **Browser scraping bank portals.** Email APIs only. Portal scraping is a fragile, ToS-fraught rabbit hole; we deliberately avoid it.

141 - **Real-time alerting / push.** Batch-pull on a schedule is sufficient.

142 - **Tax preparation, investment advice, or anything regulated.** muneem surfaces data; the user decides.

143

144 **## 7. Recommended first vertical slice**

145

146 Before generalizing, prove the end-to-end pipeline on the smallest meaningful scope:

147

148 > **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into ``transactions`` ? stored in SQLite ? a CLI command that lists last month's transactions.**

149

150 This exercises every stage exactly once, on a real input the user cares about, with no detours. It will surface the actual hard parts (which we can only guess at now) and force the open decisions in § 8 to get made on real evidence.

151

152 Concretely: pick **one** Indian bank statement format the user has many examples of (HDFC, given the user's geography and statement history), build the parser for that one format only, and put the rest of the pipeline around it. Then evaluate before generalizing.

153

154 **## 8. Open architectural decisions**

155

156 These were parked for explicit user discussion (**don't pick silently**). Items **8.1, 8.2, and 8.4** are now resolved **? see the inline resolution notes below and `[`docs/ROADMAP.md`](./docs/ROADMAP.md)` § 1. Items **8.3 and 8.5** remain open.**

157

158 **### 8.1 Language**

159

160	Option Pro Con
161	--- --- ---
162	Go only Single binary, easy distribution, great concurrency for the ingestion side, the user already uses Go for the badminton-highlight-indexer project Weak PDF parsing / OCR / ML ecosystem; rolling our own is painful
163	Python only Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF, Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype Heavier deployment story; less ergonomic for the long-running ingestion daemon
164	Go ingestion + Python parsing Plays to each language's strengths Two-language project complexity; inter-process boundary to maintain

165

166 Recommended default: **Python-only for v1**, accept the deployment-story cost in exchange for moving fast on the hard parts. Revisit if the ingestion side outgrows Python.

167

168 > **Resolved (2026-05-30): Python-only for v1.** Adopted the recommended default. See `[`docs/ROADMAP.md`](./docs/ROADMAP.md)` § 1.

169

```

170     ### 8.2 Password sourcing strategy
171
172     See § 4 Stage 2. The hybrid (dictionary ? template ? prompt) is the natural answer, but
173     the design of the rule-template config format and the candidate dictionary needs a user call.
174
175     > **Resolved (2026-05-30): Hybrid ? candidate dictionary ? per-sender rule template ?
176     interactive prompt**, caching the winning method per sender; password material never in the repo
177     or logs. The rule-template config format and dictionary design are deferred to implementation
178     (ROADMAP Milestone C.1). See [ `docs/ROADMAP.md` ](./docs/ROADMAP.md) § 1.
179
180     ### 8.3 LLM locality, per document category
181
182     The default is "local for sensitive, cloud allowed for non-sensitive after review." But
183     which models? Ollama with Llama 3 / Qwen / Mistral on the user's hardware? A small fine-tuned
184     model? Cloud (Anthropic / OpenAI / Gemini) with strict redaction for the non-sensitive
185     categories?
186
187     This decision is closely coupled to the user's hardware capacity (GPU? RAM?) and
188     willingness to wait on inference latency.
189
190     ### 8.4 First-slice scope confirmation
191
192     § 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.
193
194     > **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus
195     parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that
196     converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a
197     single end-to-end bank slice first). See [ `docs/ROADMAP.md` ](./docs/ROADMAP.md) §§ 1 and 4.
198
199     ### 8.5 Merchant canonicalization approach
200
201     Fuzzy string matching has well-known failure modes on merchant names. Do we use a
202     curated rules file (user maintains)? A library? A learned model? Defer until real data exposes
203     the actual mess.
204
205     ## 9. Proposed (not built) file layout
206
207     This is a *suggestion* for once the language decision is made. Don't materialize it
208     prematurely.
209
210     ```
211     muneem/
212     ??? README.md
213     ??? CLAUDE.md
214     ??? DESIGN.md
215     ??? docs/
216     ?   ??? prior-art-email-mining.md
217     ?   ??? (future: parser-notes-per-issuer.md, schema.md, threat-model.md)
218     ??? testdata/
219     ?   ??? promo-emails/           # non-sensitive corpus (in repo)
220     ?   ??? statements/            # GITIGNORED ? real statements, never committed
221     ??? (source tree TBD based on § 8.1)
222     ??? (DB file, GITIGNORED)
223     ```
224
225     ## 10. Glossary
226
227     - **Muneem** (?????) ? traditional Indian household/business bookkeeper.
228     - **Issuer** ? the bank, card network, broker, or merchant that produced a document.
229     - **Statement** ? a periodic financial summary from an issuer (bank statement, CC bill,
230     broker statement, MF report).
231     - **Holding** ? a position in an investment account at a point in time.
232     - **Line item** ? a single transaction or itemized purchase row within a statement or
233     receipt.
234     - **Born-digital PDF** ? a PDF generated from a software source with a real text layer

```

```

219 (most bank statements). Contrast with scanned/image PDFs.
220 ## 11. Vision continuity from Email-Mining
221
222 The predecessor project (`~/Projects/Email-Mining`) extracted promo codes and expiry
223 dates from manually-saved promo email PDFs, using PyMuPDF for text+image extraction and Gemini
224 for image OCR and summarization. Its scope was just "find the coupon I forgot about."
225
226 muneem **subsumes** that scope. The deals/coupons feature survives as **one of several
227 extraction targets** ? one of the analytics surfaces, fed by the same five-stage pipeline that
228 handles bank statements and everything else. The promo-email PDFs from that project are
229 preserved in [`testdata/promo-emails/`](./testdata/promo-emails/) as the first regression
230 corpus.
231
232 What was learned there and worth keeping: the PDF text+image extraction *shape* is right
233 (but PyMuPDF/fitz itself is AGPL-3.0 ? muneem uses pdfminer.six (MIT) instead; see § 5 rule 8);
234 image dedup by hash + SSIM is a real win for repeated chrome; the shape "page text + per-image
235 OCR ? unified text ? structured extraction" generalizes well. What we change: programmatic
236 Gmail/Outlook fetching instead of manual save-as-PDF; local extraction by default for sensitive
237 categories; persistent normalized DB instead of one-shot stdout.
238
239 Full details in [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md).
240

```

9. user (2026-05-31T14:28:01.237Z)

```

1 # Current State
2
3 > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
4 Bank)
5 > Summarizes current capabilities, completed features, and near-term next steps for
6 `muneem`.
7 > See [`ROADMAP.md`](./ROADMAP.md) for milestone tracking and [`parser-
8 architecture.md`](./parser-architecture.md) for the parser design.
9
10 > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
11 credit-card
12 > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
13 core
14 > targets**, parsed into the same local ledger, with transaction/instrument-event
15 classification to
16 > follow. The parser core is deliberately **family-agnostic** (see §2), so adding an
17 instrument
18 > family is *new data + a reconciliation rule*, not a new pipeline.
19
20 ## 1. Core Ingestion Pipeline
21 The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
22 providers (Track B plumbing):
23
24 * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
25 download implemented.
26 * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
27 attachment methods are NotImplemented (experimental).
28 * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
29 attachment/header/download NotImplemented (experimental).
30
31 Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
32 yet (see Roadmap B.4).
33
34 ## 2. Statement Parsers (concept-core architecture)
35 Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
36 by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
37 (`_autoload_parsers`), so the registry is populated in the production path (not just under
38 tests).
39

```

25 The parser core is **concept-first and verification-first** (see [`parser-architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a shared **verifier** (`validation.py`) decides trust by **arithmetic** ? per-row balance walk + total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future **instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as **data, not code**. Bank **savings** parsers share one engine-driven path ? `savings.py` (per-table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly a profile; HDFC keeps bespoke coordinate clustering (the documented `extract_tables` exception).

26

27 **Active issuers** ? we have real statements + passwords, so these get validated on real data:

28

29 * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate clustering** (deliberately **not** `extract_tables()` ? HDFC's table is semi-ruled). **Self-validating**: running-balance reconciliation + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records `documents.status` = `'reconciled'/'unreconciled'`. Refactored onto the concept core as the **reference implementation**. ? **Verified on the user's 6 real statements** ? all reconcile **and** bookend, without inspecting any value.

30 * **Axis Bank** (`axis.py`): Savings + Credit Card. Savings runs the shared engine-driven path (`savings.py`) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a **balance-delta reconstruction**: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks `amount == |?balance|`. Oracle-gated, so it never persists unreconciled rows. ? **6/7 real statements reconcile + persist**; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores **nothing**). Credit-card has no running balance (count only).

31 * **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just `AUBANK_PROFILE`. ? **4/4 real statements reconcile + persist** (the cleanest issuer so far).

32

33 > **Dropped: ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

34

35 All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?3 have landed** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution** is now wired into `muneem ingest --doc-type bank_statement` ? the hybrid unlock (keychain-cached + `password_rules.yaml` candidates ? interactive prompt; `unlock.py` + `statement_unlock.py`) runs before extraction and forwards the password to the parser, in-memory only. See [`ingesting-statements.md`](./ingesting-statements.md).

36

37 See `password_rules.py` (prefers user config dir) and `cli.py:ingest`.

38

39 **## 3. Database & CLI**

40 * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_*`, `hdfc_transactions`). Dedup on sha256. No versioned migration scripts yet (IF NOT EXISTS + user_version pragma). A unified `transactions` table (plus `holdings` for instruments) is planned once the per-bank schemas stabilize (parser-architecture.md §9).

41 * CLI: `ingest <pdf>` (promo, or `--doc-type bank_statement` ? **auto-unlocks encrypted PDFs**, see [`ingesting-statements.md`](./ingesting-statements.md)), `offers list`, `txns list <bank>`, and `eval <folder>` (reconcile-rate report over a local statement folder ? the seed?golden onboarding loop, see [`onboarding-a-bank.md`](./onboarding-a-bank.md)).

42 * Config uses `platformdirs` for local dirs; secrets via keyring.

43

44 **## 4. Testing & Hygiene**

45 * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine

```

fixtures (default CI).
46      * Security guards in `test_repo_hygiene.py` + license allowlist test.
47      * Local-only tests for real statements (gitignored data; passwords via env/keyring,
never committed).
48
49      ## 5. Next Steps
50      Concept-core steps 1?3 are done, and password unlock is now wired into `muneem
ingest` (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: HDFC 6/6, AU 4/4, Axis
6/7 (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
51
52      * Recover the May 2025 vintage: the clusterer drops one transaction line ?
incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing;
June 2025 was recovered via balance-delta reconstruction).
53      * Step 4 ? L3 local-vision fallback (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
54      * Step 5 ? unify storage (`transactions` + `holdings`) and begin the instrument
family (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
55      * Regression CI for statements (decided 2026-06, deferred ? not yet built): a
smart mix of synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the
real HDFC/AU statements (which stay local, off GitHub). Grounded plan + rationale in
[ROADMAP.md](./ROADMAP.md) ($ "Planned ? Regression CI for statement parsing").
56
57      Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization), encryption-at-rest timing (D.2).
58
59      The project stays local-first, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
60

```

10. user (2026-05-31T14:28:01.278Z)

```

1      # muneem ? Dependency policy & license allowlist
2
3      muneem must be safe to use commercially and must minimize paid dependencies.
4      These are standing principles (DESIGN.md § 5, rules 8?9). This doc is the allowlist;
5      `tests/test_dependency_licenses.py` enforces it (a dependency not listed here fails CI).
6
7      ## Rules
8
9      1. Only permissive, commercial-safe licenses: MIT, BSD (2/3-clause), Apache-2.0, ISC,
HPND.
10     2. Banned: GPL, LGPL, AGPL, and source-available / non-commercial licenses.
11         ? PyMuPDF / `fitz` is AGPL-3.0 ? do not use it. (Its own metadata reads
12         "Dual Licensed - GNU AFFERO GPL 3.0 or Artifex Commercial"; the non-AGPL option
13         is a paid license, which also violates rule 4.)
14     3. MPL-2.0 (weak, file-level copyleft) only as a fallback when no permissive option
15         exists, and flagged here.
16     4. Prefer local / self-hosted / open-source runtimes over paid APIs. Paid APIs
17         (cloud LLMs, paid OCR/vision) are a fallback only, gated behind explicit
approval.
18         We are willing to train our own image/OCR models rather than take a paid
dependency.
19     5. Adding a dependency requires updating both this table and the allowlist in
20         `tests/test_dependency_licenses.py` ? otherwise the build fails.
21
22     ## Current dependencies
23
24     | Package | License | Role |
25     |---|---|---|
26     | typer | MIT | CLI |
27     | platformdirs | MIT* | OS config/data dirs |
28     | keyring | MIT | OS keychain (secrets) |
29     | pdfminer.six | MIT | PDF text extraction (replaces AGPL PyMuPDF) |

```

```

30 | pypdfium2 | Apache-2.0 / BSD-3 | PDF page rendering for OCR |
31 | pytesseract | Apache-2.0 | local OCR (Tesseract) wrapper |
32 | Pillow | HPND | image handling for OCR |
33 | pypdf | BSD-3-Clause | decrypt password-protected statements (Stage 2) |
34 | google-api-python-client | Apache-2.0 | Gmail API (read-only) |
35 | google-auth | Apache-2.0 | OAuth credentials |
36 | google-auth-oauthlib | Apache-2.0 | installed-app OAuth consent flow |
37 | msal | MIT | Microsoft Graph API OAuth |
38 | requests | Apache-2.0 | HTTP library for Outlook provider |
39 | pdfplumber | MIT | text/table + coordinate extraction for statements (HDFC/Axis/AU) |
40 | pytest *(dev)* | MIT | tests |
41 | ruff *(dev)* | MIT | lint / format |
42 | PyYAML *(dev)* | MIT* | read the test answer key |
43
44 \* PyMuPDF (AGPL), PyYAML (MIT) and platformdirs (MIT) were confirmed directly from
45 installed package metadata; the rest are MIT per their upstream projects.
46
47 ## System tools (not Python packages)
48
49 - **Tesseract OCR** (Apache-2.0) ? local OCR engine, invoked via `pytesseract`. Install
50 separately (e.g. `winget install UB-Mannheim.TesseractOCR`). muneem finds it on PATH,
51 at the Windows default location, or via the `MUNEEM_TESSERACT_CMD` env var.
52
53 ## Current / recently added (permissive)
54
55 - **pypdf** (BSD-3) ? PDF decryption (Stage 2, used by unlock.py and parsers). Chosen
over
56 pikepdf (MPL-2.0 flagged fallback) for license simplicity.
57 - **scikit-image / numpy** (BSD) ? planned for image dedup (SSIM) in future; not yet in
58 pyproject (Pillow covers basic image needs for OCR today).
59
60 ## Planned (future, permissive, no paid)
61
62 - A **purpose-trained OCR model** for stylized promo codes (Tesseract is approximate on
63 some banners like Swiggy). Aligns with "train our own" (DESIGN §5 rule 9).
64
65 **Optional deps** (outlook extra): msal + requests ? only loaded when OutlookProvider is
66 used; guarded by ImportError in code.
67

```

11. assistant (2026-05-31T14:28:03.506Z)

Perfect. Now let me search for database-related code and then conduct a comprehensive research of how other tools handle encryption:

12. user (2026-05-31T14:28:04.705Z)

```

C:\Users\avidu\Projects\muneem\src\muneem\cli.py
C:\Users\avidu\Projects\muneem\src\muneem\config.py
C:\Users\avidu\Projects\muneem\src\muneem\db.py
C:\Users\avidu\Projects\muneem\src\muneem\extract.py
C:\Users\avidu\Projects\muneem\src\muneem\ingestion\base.py
C:\Users\avidu\Projects\muneem\src\muneem\ingestion\gmail.py
C:\Users\avidu\Projects\muneem\src\muneem\ingestion\imap.py
C:\Users\avidu\Projects\muneem\src\muneem\ingestion\outlook.py
C:\Users\avidu\Projects\muneem\src\muneem\ingestion\__init__.py
C:\Users\avidu\Projects\muneem\src\muneem\ocr.py
C:\Users\avidu\Projects\muneem\src\muneem\offers.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\au.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\base.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\clustering.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\concepts.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py
C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py

```

```
C:\Users\avidu\Projects\muneem/src/muneem/parsers/profiles.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/savings.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/validation.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/__init__.py
C:\Users\avidu\Projects\muneem/src/muneem/password_rules.py
C:\Users\avidu\Projects\muneem/src/muneem/secrets.py
C:\Users\avidu\Projects\muneem/src/muneem/statement_unlock.py
C:\Users\avidu\Projects\muneem/src/muneem/unlock.py
C:\Users\avidu\Projects\muneem/src/muneem/__init__.py
```

13. user (2026-05-31T14:28:05.015Z)

Found 20 files

```
src\muneem\cli.py
tests\test_regression_synth.py
.githubhooks\precommit_check.py
tests\test_cli.py
tests\test_precommit_hook.py
src\muneem\db.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
tests\parsers\test_au.py
tests\parsers\test_local_au.py
tests\parsers\test_axis.py
tests\parsers\test_local_axis.py
tests\parsers\test_hdfc.py
tests\parsers\test_validation.py
src\muneem\parsers\hdfc.py
src\muneem\parsers\concepts.py
tests\test_repo_hygiene.py
src\muneem\parsers\base.py
tests\test_db.py
src\muneem\config.py
```

14. assistant (2026-05-31T14:28:07.468Z)

Now let me read the database code to understand the current schema:

15. user (2026-05-31T14:28:07.478Z)

```
1      """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4      (accounts, transactions, holdings, merchants, items) grows with later milestones.
5      ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7      """
8
9      from __future__ import annotations
10
11      import sqlite3
12      from collections.abc import Iterator
13      from contextlib import contextmanager
14      from pathlib import Path
15
16      SCHEMA_VERSION = 4
17
18      _SCHEMA = """
19      CREATE TABLE IF NOT EXISTS documents (
20          id            INTEGER PRIMARY KEY,
21          source        TEXT NOT NULL,
22          external_id   TEXT,
23          sender        TEXT,
24          subject       TEXT,
```

```

25         received_date TEXT,
26         doc_type      TEXT,
27         sha256        TEXT NOT NULL UNIQUE,
28         path          TEXT,
29         status        TEXT NOT NULL DEFAULT 'parsed',
30         created_at    TEXT NOT NULL DEFAULT (datetime('now'))
31     );
32
33     CREATE TABLE IF NOT EXISTS offers (
34         id              INTEGER PRIMARY KEY,
35         document_id    INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
36         merchant       TEXT,
37         promo_code     TEXT,
38         description    TEXT,
39         expiry_raw     TEXT,
40         expiry_type    TEXT,
41         source         TEXT NOT NULL DEFAULT 'text',
42         redeemed       INTEGER NOT NULL DEFAULT 0,
43         source_text    TEXT
44     );
45
46     CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
47
48     -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
49     -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
50     -- Normalization into a unified view layer is planned for a later phase
51     -- once all bank schemas are stabilized.
52     CREATE TABLE IF NOT EXISTS axis_transactions (
53         id INTEGER PRIMARY KEY,
54         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
55         account_number TEXT,
56         txn_date TEXT,
57         value_date TEXT,
58         particulars TEXT,
59         debit REAL,
60         credit REAL,
61         balance REAL
62     );
63
64     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
65         id INTEGER PRIMARY KEY,
66         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
67         account_number TEXT,
68         txn_date TEXT,
69         particulars TEXT,
70         merchant_category TEXT,
71         amount REAL,
72         type TEXT
73     );
74
75     CREATE TABLE IF NOT EXISTS au_transactions (
76         id INTEGER PRIMARY KEY,
77         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
78         account_number TEXT,
79         txn_date TEXT,
80         value_date TEXT,
81         particulars TEXT,
82         debit REAL,
83         credit REAL,
84         balance REAL
85     );
86
87     CREATE TABLE IF NOT EXISTS hdfc_transactions (
88         id INTEGER PRIMARY KEY,
89         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,

```



```

90         account_number TEXT,
91         txn_date TEXT,
92         value_date TEXT,
93         particulars TEXT,
94         ref TEXT,
95         withdrawal REAL,
96         deposit REAL,
97         closing_balance REAL
98     );
99     """
100
101
102     @contextmanager
103     def connect(db_path: Path | str) -> Iterator[sqlite3.Connection]:
104         """Open a connection (commit on success, always close). Parent dirs created."""
105         path = str(db_path)
106         if path != ":memory:":
107             Path(path).parent.mkdir(parents=True, exist_ok=True)
108         conn = sqlite3.connect(path)
109         conn.row_factory = sqlite3.Row
110         conn.execute("PRAGMA foreign_keys = ON")
111         try:
112             yield conn
113             conn.commit()
114         finally:
115             conn.close()
116
117
118     def init_db(conn: sqlite3.Connection) -> None:
119         """Create tables if absent and stamp the schema version."""
120         conn.executescript(_SCHEMA)
121         conn.execute(f"PRAGMA user_version = {SCHEMA_VERSION}")
122
123
124     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
125         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
126 (sha256,)).fetchone()
127
128
129     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
130         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
131 txns."""
132         cols = ", ".join(fields)
133         placeholders = ", ".join("?" for _ in fields)
134         cur = conn.execute(
135             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
136             tuple(fields.values()),
137         )
138         return int(cur.lastrowid)
139
140
141     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
142         """Insert into documents table. Delegates to the shared helper for DRY."""
143         return _insert_row(conn, "documents", dict(fields))
144
145
146     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
147         return _insert_row(conn, "offers", fields)
148
149
150     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
151         return conn.execute(
152             """
153             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
154             d.external_id, d.path
155         """

```

```

153         FROM offers o
154         JOIN documents d ON d.id = o.document_id
155         ORDER BY o.id
156         """
157     ).fetchall()
158
159
160     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
161         return _insert_row(conn, "axis_transactions", fields)
162
163
164     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
165         return _insert_row(conn, "axis_cc_transactions", fields)
166
167
168     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
169         return conn.execute(
170             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
171         ).fetchall()
172
173
174     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
175         return conn.execute(
176             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
177         ).fetchall()
178
179
180     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
181         return _insert_row(conn, "hdfc_transactions", fields)
182
183
184     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
185         return conn.execute(
186             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
187         ).fetchall()
188
189
190     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
191         return _insert_row(conn, "au_transactions", fields)
192
193
194     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
195         return conn.execute(
196             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
197         ).fetchall()
198

```

16. user (2026-05-31T14:28:31.112Z)

```

.venv\Lib\site-packages\cryptography\fernet.py:         self._encryption_key = key[16:]
.venv\Lib\site-packages\cryptography\fernet.py:     def encrypt(self, data: bytes) -> bytes:
.venv\Lib\site-packages\cryptography\fernet.py:         return self.encrypt_at_time(data,
int(time.time()))
.venv\Lib\site-packages\cryptography\fernet.py:     def encrypt_at_time(self, data: bytes,
current_time: int) -> bytes:
.venv\Lib\site-packages\cryptography\fernet.py:         return self._encrypt_from_parts(data,
current_time, iv)
.venv\Lib\site-packages\cryptography\fernet.py:     def _encrypt_from_parts(
.venv\Lib\site-packages\cryptography\fernet.py:         encryptor = Cipher(
.venv\Lib\site-packages\cryptography\fernet.py:             algorithms.AES(self._encryption_key),
.venv\Lib\site-packages\cryptography\fernet.py:         ).encryptor()

```

```
.venv/Lib/site-packages/cryptography/fernet.py:      ciphertext =
encryptor.update(padded_data) + encryptor.finalize()
.venv/Lib/site-packages/cryptography/fernet.py:      algorithms.AES(self._encryption_key),
modes.CBC(iv)
.venv/Lib/site-packages/cryptography/fernet.py:      def encrypt(self, msg: bytes) -> bytes:
.venv/Lib/site-packages/cryptography/fernet.py:      return self.encrypt_at_time(msg,
int(time.time()))
.venv/Lib/site-packages/cryptography/fernet.py:      def encrypt_at_time(self, msg: bytes,
current_time: int) -> bytes:
.venv/Lib/site-packages/cryptography/fernet.py:      return
self._fernets[0].encrypt_at_time(msg, current_time)
.venv/Lib/site-packages/cryptography/fernet.py:      return
self._fernets[0]._encrypt_from_parts(p, timestamp, iv)
.venv/Lib/site-packages/cryptography/hazmat/backends/openssl/backend.py:      # TripleDES
encryption is disallowed/deprecated throughout 2023 in
.venv/Lib/site-packages/cryptography/hazmat/backends/openssl/backend.py:      def
rsa_encryption_supported(self, padding: AsymmetricPadding) -> bool:
.venv/Lib/site-packages/cryptography/hazmat/primitives/asymmetric/dh.py:
encryption_algorithm: _serialization.KeySerializationEncryption,
.venv/Lib/site-packages/cryptography/hazmat/primitives/asymmetric/dsa.py:
encryption_algorithm: _serialization.KeySerializationEncryption,
```

17. assistant (2026-05-31T14:28:33.325Z)

Now let me conduct comprehensive research on encryption practices across personal finance tools and the security options for SQLite:

18. user (2026-05-31T14:28:34.970Z)

Launching skill: deep-research

19. user (2026-05-31T14:28:34.969Z)

Run the "deep-research" workflow.

Deep research harness ? fan-out web searches, fetch sources, adversarially verify claims, synthesize a cited report.

When the user wants a deep, multi-source, fact-checked research report on any topic. BEFORE invoking, check if the question is specific enough to research directly ? if underspecified (e.g., "what car to buy" without budget/use-case/region), ask 2-3 clarifying questions to narrow scope. Then pass the refined question as args, weaving the answers in.

Phases:

- Scope: Decompose question (from args) into 5 search angles
- Search: 5 parallel WebSearch agents, one per angle
- Fetch: URL-dedup, fetch top 15 sources, extract falsifiable claims
- Verify: 3-vote adversarial verification per claim (need 2/3 refutes to kill)
- Synthesize: Merge semantic dupes, rank by confidence, cite sources

Invoke: Workflow({ name: "deep-research", args: "How do local-first personal finance tools (Actual Budget, Firefly III, GnuCash, beancount, ledger-cli, Maybe) handle database encryption at rest? What are the operational details of SQLCipher vs SQLite3MC (backups, migrations, WAL, corruption risk), and are there security advisories? What is a pragmatic phased approach to encryption for a personal finance tool just starting to store real numbers?" })

20. assistant (2026-05-31T14:28:36.929Z)

I'll run the deep-research workflow to get comprehensive, verified information on encryption practices for local-first finance tools and SQLite encryption options.

21. user (2026-05-31T14:28:38.005Z)

{

"name": "README.md",
"path": "README.md",
"sha": "4886a8eb80368d071b76f03f1726daca6d1406da",
"size": 3986,
"url": "https://api.github.com/repos/actualbudget/actual/contents/README.md?ref=master",
"html_url": "https://github.com/actualbudget/actual/blob/master/README.md",
"git_url": "https://api.github.com/repos/actualbudget/actual/git/blobs/4886a8eb80368d071b76f03f1726daca6d1406da",
"download_url": "https://raw.githubusercontent.com/actualbudget/actual/master/README.md",
"type": "file",
"content": "PHAgYwpxZ249ImNlbnRlciI+CiAgPGltZyBzcmM9Ii9kZW1vLnBuZyIgYWx0\n\nPSJB3R1YWxidWRnZXQiIC8+CjwvcD4KCimjIEldHRpbmcgU3Rhc nRlZAoK\n\nnQWN0dWFsIGlzIGlGEgB9jYwWtZmlyc3QgcGVyc29uYWwgZm luYW5jZS
B0b29s\n\nnLiBjDCBpcyAxMDA lIGZyZWUgYW5kIG9wZW4tc291cmNlLCB3cm l0dGVuIGlu\n\nnIE5vZGVKUywg aXQgaGFzIGlGEgc3luY2hyb25pemF0aW9uIGVsZW1lbnQgc28g\n\nndGhhdCBhbGwgew91ciBjaGFuZ2VzIGNhbiBtb3ZlIGJldHdlZW4gZGV2aWNL\n\n\ncyB3aXRob3V0IGFueSBoZWZ2eSBsaW0aW5nLgoKSWYgew91IGFyZSBpb nRl\n\n\ncmVzdGVkIGluIGNvbnRyaWJ1dGluZywg b3Igd2FudCB0byBrbm93IGhvd yBk\n\n\nZXZlbG9wbWVudCB3b3Jrcywg c2VlIG91ciBbY29udHJpYnV0aW5nXShodHRw\n\n\nczovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcy9jb250cm lidXRpbmcvKSbk2N1\n\n\nnbWVudCB3ZSB3b3VsZCBsb3ZlIHRvIGhhd mUgew91LgoKV2FudCB0byBzYXkg\n\n\nndGhhbmtzPyBD bGljayB0aGUg4q2QIGF0IHRoZSB0b3Agb2YgdGh lIHBhZ2Uu\n\n\nnCgojIyBLZX kgTGlua3MKCi0gQWN0dWFsIFtk aXNjb3JkXShodHRwc zovL2R p\n\n\n\nc2Nvc mQuZ2cvcFJZTlly nF c1QSkgy29tbXVuaXR5LgotIEFjdHVhbCBbQ29t\n\n\n\nnbXVuaXR5IERvY3VtZW50YXRpb25dKGh0dHBzOi8vYWN0dWFsYnV kZ2V0Lm9y\n\n\n\nnZy9kb2NzKQotIFtG cmVxdWVudGx5IGFza2VkIH F1ZXN0aW9uc10oaHR0cHM6\n\n\n\nnLy9hY3R1YWxidWRnZXQub3JnL2RvY3MvZmFkXQoKIyMgSW5zdG FsbGF0aW9u\n\n\n\nnCgpUaGVyZSBhcmUgZm91ciB3YXZlIHRvIGRlcGxveSBBY3R1YwW6CgoxLiBP\n\n\n\nnbmUtY2xpY2sgZGVwbG95bW VudCBbdm lHfBpa2FQb2RzXShodHRwc zovL3d3\n\n\n\nndy5waWthcG9kcy5jb20vcG9kc z9ydW49YWN0dWFsKSAofjEuNDAGJC9t b250\n\n\n\n\naCk gLSByZWVwbW1lbmRlZCBmb3Igbm9uLXRlY2huaWNhbCB1c2VycwoxLiBN\n\n\n\nnYw5hZ2VkiGhvc3RpbmcgW3ZpYSBG bHkuaw9dKGh0dHBzOi8vYWN0dWFsYnV k\n\n\n\nnZ2V0Lm9yZy9kb2NzL2Luc3RhbGwvZmx5KSAofjEuNTAGJC9tb250aCkKMS4g\n\n\n\n\nnU2VsZi1ob3N0ZWQgYnkgdXNpbmcgW2EgRG9ja2VyIGltYw dLXShodHRwc zov\n\n\n\n\nnL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcy9pb nN0YwxsL2RvY2t lcikKMS4gTG9j\n\n\n\n\nnYwWtb25seSBhcHBzIC0gW2Rvd25sb2FkYwJsZSBXaw5kb3dzLCBNYwMgYw5k\n\n\n\n\nnIExpbnV4IGFwcHNdKGh0dHBzOi8vYWN0dWFsYnV kZ2V0Lm9yZy9kb3dubG9h\n\n\n\n\nnZC8pIHlvdSBjYw4gc nVuIG9uIHlvdX IgZGV2aWNL CgpMZWFybiBtb3JlIGlu\n\n\n\n\nnIHRoZSBbaW5zdGFsbGF0aW9uIG luc3RydWN0aW9ucyBkb2NzXShodHRwc zov\n\n\n\n\n\nnL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcy9pb nN0YwxsLykuCgojIyBSZW FkeSB0\n\n\n\n\n\nnbYBTdGFydCBCdWRnZXRp bmc/CgpSZWFkIGFib3V0IFtFbn ZlbG9wZSBidWRn\n\n\n\n\n\nnZXRp bmd dKGh0dHBzOi8vYWN0dWFsYnV kZ2V0Lm9yZy9kb2NzL2ldHRpbmct\n\n\n\n\n\n\nc3Rhc nRlZC9lbnZlbg9wZS1idWRnZXRp bmc pIHRvIGtub3cgbW9yZSBhYm91\n\n\n\n\n\nndCB0aGUgaWRlYSBiZW hpbmQgQWN0dWFsIEJ1ZGdlcD4KCimjIyBB cmUgew91\n\n\n\n\n\nnIG5ldyB0byBidWRnZXRp bmcgb3Igd2FudCB0byBzdGFydCBmcmVzaD8KCKNo\n\n\n\n\n\nnZWNRIG91dCB0aGUgY29tbXV uaxR5J3MgW1N0YXJ0aW5nIEZyZXNoXShodHRw\n\n\n\n\n\n\nczovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm l0eSBE\nnb2N1bWVudGF0aW9uXShodHRwc zovL2FjdHVhbGJ1ZGdlc5vcmcvZG9jcyks\nnIHRoaXMgaW5jbH YkZXMGdG9waWNLIG9uIEJ1ZGdlc5vcmcvZG9jcy9nZXR0aW5nLXN0YXJ0ZWQvc3Rh\nncnRpbmctZnJlc2gpIGd1aWRlIHNVlHlvdSBjYw4gcXVpY2tseSBnZXQgdXAg\nnYw5kIHJ1bm5pbmchCgojIyMgQXJlIHlvdSBtaWdyYXRpbmcgZnJvbSBvdGhl\nnciBidWRnZXRp bmcgYXBwc28KCKNoZWNRIG91dCB0aGUgY29tbXVuaXR5J3Mg\nnw01pZ3JhdGlvb l0oaHR0cHM6Ly9hY3R1YWxidWRnZXQub3JnL2RvY3Mvbw l\ncmF0aW9uLykgZ3VpZGUGdG8gc3Rhc nQganVt cGluZyBvbiB0aGUgQWN0dWFs\nnIEJ1ZGdlcD4KCimjIyIERvY3VtZW50YXRpb24KCl d lIGhhd mUgYSB3\nnawRlIHJh bmdlIG9mIGRvY3VtZW50YXRpb24gb24gaG93IHRvIHVzZSBBY3R1\nnYwW sIHRoaXMgaXMGYwxsIGF2YwlsYwJsZSBpb iBvdX IgW0NvbW11bm

```
Vk\nZ2V0IHBvc3NpYmxlIQoKPGEGaHJLZj0iaHR0cHM6Ly93d3cubmV0bGlmeS5j\nnb20iPiA8aW1nIHNYZ0iaHR0cHM6Ly93d3cubmV0bGlmeS5jb20vdjMvaW1n\nnL2NvbXBvbmVudHMvbmV0bGlmeS1jb2xvci1hY2NlbnQuc3ZnIiBhbHQ9IkRl\ncGxveXMgYnkgTmV0bGlmeSIgLz4gPC9hPgo=\n",
"encoding": "base64",
"_links": {
  "self": "https://api.github.com/repos/actualbudget/actual/contents/README.md?ref=master",
  "git": "https://api.github.com/repos/actualbudget/actual/git/blobs/4886a8eb80368d071b76f03f1726daca6d1406da",
  "html": "https://github.com/actualbudget/actual/blob/master/README.md"
}
}
```

22. assistant (2026-05-31T14:28:39.878Z)

Let me use WebSearch to conduct the research more efficiently:

23. user (2026-05-31T14:28:46.365Z)

Web search results for query: "Actual Budget local-first encryption database at rest"

Links: [{"title":"Hosted Actual Budget ? Self-Hosted YNAB Alternative | ElfHosted","url":"https://store.elfhosted.com/product/actual-budget/"},"{"title":"Deploy Actual Budget | Open Source YNAB Alternative","url":"https://railway.com/deploy/actual-personal-finance"},"{"title":"Settings | Actual Budget","url":"https://actualbudget.org/docs/settings/"},"{"title":"Your Finances ? made simple | Actual Budget","url":"https://actualbudget.com/"},"{"title":"Configuring the Server | Actual Budget","url":"https://actualbudget.org/docs/config/"},"{"title":"Syncing Across Devices | Actual Budget","url":"https://actualbudget.org/docs/getting-started/sync/"},"{"title":"Actual Budget | Coolify Docs","url":"https://coolify.io/docs/services/actualbudget"},"{"title":"Frequently Asked Questions | Actual Budget","url":"https://actualbudget.org/docs/faq/"}]

Based on the search results, here's information about Actual Budget's local-first architecture and encryption:

Architecture

Actual Budget is an open-source, local-first, end-to-end-encrypted application. The architecture is local-first, so the web app and mobile apps work fully offline, with sync happening through the Actual Server.

Database Storage

The application deploys with a persistent volume for SQLite storage, with no external database needed ? everything runs in one lightweight service. Budget data is stored as SQLite files.

Encryption

Transaction data is encrypted client-side before reaching the server, end-to-end ? meaning even the hosting provider can't read it. End-to-end encryption is preserved ? the host stores the ciphertext while the keys never leave client devices.

End-to-end encryption offers the ability to generate a key based on a password that encrypts data so hosted services can't read budget data. End-to-end encryption allows you to encrypt your budget data on the remote server with a password, which you can enable if you don't trust the server's owners.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. user (2026-05-31T14:28:47.245Z)

Web search results for query: "Firefly III GnuCash database encryption"

Links: [{"title":"Day 39: Firefly III - Free and Open Source Personal Finance Manager ? 7 Days of Docker - portalZINE.DE","url":"https://portalzine.de/day-39-firefly-iii-free-and-open-source-personal-finance-manager-7-days-of-docker/"},"{"title":"Security - Firefly III documentation","url":"https://docs.firefly-iii.org/explanation/more-information/security/"},"{"title":"General questions - Firefly III documentation","url":"https://docs.firefly-iii.org/references/faq/firefly-iii/general/"},"{"title":"Changing the app key (re-encrypting the data in the database) · Issue #1095 · firefly-iii/firefly-iii","url":"https://github.com/firefly-iii/firefly-iii/issues/1095"},"{"title":"Firefly III ? Self-Hosted Personal Finance Manager","url":"https://mymoneykeeper.com/program/firefly-iii/"},"{"title":"Firefly III | Secure Budgeting | Finance | Awesome Privacy","url":"https://awesome-privacy.xyz/finance/secure-budgeting/firefly-iii"},"{"title":"Secure Budgeting | Awesome Privacy","url":"https://awesome-privacy.xyz/finance/secure-budgeting/"},"{"title":"GnuCash","url":"https://brianlovin.com/hn/31219754"},"{"title":"Compare Firefly III vs. GnuCash in 2026","url":"https://slashdot.org/software/comparison/Firefly-III-vs-GnuCash/"}]

Based on the search results, here's what I found regarding Firefly III and GnuCash database encryption:

Firefly III Database Encryption

The database is not encrypted, and data in the tables of the database is readable for humans. However, descriptions and other values were encrypted using AES and decrypted transparently, which adds security because if the database is taken, the values are unreadable without the key. That said, this is no longer the case in current versions.

Regarding data-at-rest security, no encryption of data-at-rest is used, so users should use their operating system's disk encryption features for further protection of data-at-rest.

GnuCash

The search results provide information comparing Firefly III and GnuCash as tools but do not contain specific details about GnuCash's database encryption methods. The results indicate that GnuCash and Money Manager Ex store data in portable files (SQLite, XML) that you can copy to external drives or cloud storage.

For Firefly III security beyond database encryption, users should be aware that a strong password to authenticate to the database and a dedicated user for the Firefly III database is recommended. Additionally, 2FA is a feature of Firefly III that users should consider enabling.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. user (2026-05-31T14:28:47.334Z)

Web search results for query: "SQLCipher vs SQLite3MC encryption security advisories CVE"

Links: [{"title":"CVE-2020-27207 : Zetetic SQLCipher 4.x before 4.4.1 has a use-after-free, related to sqlcipher_co","url":"https://www.cvedetails.com/cve/CVE-2020-27207/"},"{"title":"Update for SQLCipher with SQLite3 version 3.40.1 · Issue #464 · sqlcipher/sqlcipher","url":"https://github.com/sqlcipher/sqlcipher/issues/464"},"{"title":"GitHub - sqlcipher/sqlcipher: SQLCipher is a standalone fork of SQLite that adds 256 bit AES encryption of database files and other security features. · GitHub","url":"https://github.com/sqlcipher/sqlcipher"},"{"title":"SQLCipher ? Full Database Encryption for SQLite | Hacker News","url":"https://news.ycombinator.com/item?id=9019836"},"{"title":"New critical denial of service vulnerability - Telekom Security","url":"https://github.security.telekom.com/assets/advisories/dl-210308-critical-dos-vulnerability-in-sqlcipher-sql-command-processing-data.pdf"},"{"title":"Popular SQL Server Database Encryption Choices","url":"https://arxiv.org/pdf/1901.03179"},"{"title":"Zetetic SQLcipher : Security vulnerabilities, CVEs Code

```
Execution", "url": "https://www.cvedetails.com/vulnerability-list/vendor_id-23070/product_id-82162/opec-1/Zetetic-Sqlcipher.html"}, {"title": "Equivalence-based Security for Querying Encrypted Databases: Theory and Application to Privacy Policy Audits", "url": "https://arxiv.org/pdf/1508.02448"}, {"title": "Securing SQL based databases with cryptographic protocols", "url": "https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/10664604"}]
```

Based on the search results, I found information about SQLCipher security advisories and CVEs, though the results did not contain specific information comparing SQLCipher with SQLite3MC. Here's what I found:

SQLCipher Security Advisories and CVEs

Zetetic SQLCipher 4.x before 4.4.1 has a use-after-free vulnerability related to `sqlcipher_codec_pragma` and `sqlite3Strlen30` in `sqlite3.c`, which can allow a remote denial of service attack. A SQL injection can be used to execute crafted SQL command sequences to trigger this vulnerability.

Additionally, SQLCipher version 4.5.3 was identified as vulnerable through its use of SQLite3 version 3.39.2, with the same vulnerability fixed in SQLite3 version 3.40.1.

Overview of SQLCipher

SQLCipher is a standalone fork of the SQLite database library that adds 256 bit AES encryption of database files and other security features. SQLCipher is maintained by Zetetic, LLC.

Note: The search results did not contain specific information about SQLite3MC or a direct comparison between SQLCipher and SQLite3MC encryption security advisories. To get a more detailed comparison of the two tools and any specific CVEs associated with SQLite3MC, you may need to conduct a more targeted search specifically for SQLite3MC security information.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. user (2026-05-31T14:28:48.877Z)

Web search results for query: "beancount ledger-cli plaintext database storage"

Links: [{"title": "Plain Text Accounting - plaintextaccounting.org", "url": "https://plaintextaccounting.org/"}, {"title": "Beancount.io - Plain-Text Accounting. Powerful, Precise, Auditable.", "url": "https://beancount.io/blog/2025/04/15/beancount-ecosystem"}, {"title": "Accounting for the simple minds with plain text accounting", "url": "https://dustri.org/b/accounting-for-the-simple-minds-with-plain-text-accounting.html"}, {"title": "A Query Language for Beancount", "url": "https://ledger-cli.narkive.com/GEHM5Wto/a-query-language-for-beancount"}, {"title": "Beancount's Technical Edge vs. Ledger, hledger, and GnuCash - Beancount.io", "url": "https://beancount.io/blog/2025/07/22/beancounts-technical-edge-a-deep-dive-on-performance-python-api-and-data-integrity-vs-ledger-hledger-and-gnucash"}, {"title": "Beancount Basics | The Bean Ledger", "url": "https://thebeanledger.com/posts/basics/"}, {"title": "Ledger CLI - beancount", "url": "https://paisa.fyi/reference/ledger-cli/"}, {"title": "I'm a big fan of CLI accounting software (Ledger, Beancount and hledger) for my ... | Hacker News", "url": "https://news.ycombinator.com/item?id=16859341"}, {"title": "hledger and Beancount - hledger", "url": "https://hledger.org/beancount.html"}]

Based on the search results, here's what I found about Beancount, Ledger-CLI, and plaintext database storage:

Overview

Beancount belongs to the family of plain text double-entry accounting tools, among which Ledger CLI (John Wiegley's Ledger) and hledger are prominent. While all these systems share the core

idea of plaintext ledger files and double-entry bookkeeping, they differ in syntax, philosophy, and ecosystem maturity.

Plaintext Storage

Beancount utilizes plain text files as its database, allowing all transactions to be easily read and modified with any text editor. Its storage format is plain-text: easy to edit, easy to backup, easy to read, easy to version, easy to parse, hard to fuck up.

Key Features

Beancount includes a powerful query language (inspired by SQL) to slice and dice the ledger. The bean-query tool treats the parsed ledger as a database and lets you run analytical queries on it. Additionally, users can write plugins in Python that operate on the stream of transactions (for example, to enforce a custom rule or generate automatic entries). These plugins run as the file is processed, effectively extending Beancount's core functionality.

Comparison Notes

Beancount differentiates itself with its emphasis on strictness, plugin-based extensibility, and a user-friendly web interface. Ledger remains the classic, highly flexible tool favored by command-line purists and those who need ultimate speed (Ledger's C++ engine is very fast on huge files). hledger provides a middle ground ? much of Ledger's functionality with a bit more structure and an officially supported (if simple) web UI.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. user (2026-05-31T14:28:54.305Z)

Web search results for query: "personal finance app encryption strategy when to enable"

Links: [{"title":"How to Protect Your Data on Money and Budget Apps | Equifax","url":"https://www.equifax.com/personal/education/identity-theft/articles/-/learn/apps-cybersecurity-tips/"},"{"title":"5 Security Features You Should Look For When Choosing a Financial App","url":"https://www.wisdomtreeprime.com/blog/5-security-features-you-should-look-for-when-choosing-a-financial-app/"},"{"title":"Best Strategies for App Configuration Data Protection in Financial Android Apps | by amol pawar | softAai Blogs | Medium","url":"https://medium.com/softaai-blogs/best-strategies-for-app-configuration-data-protection-in-financial-android-apps-db48058c1bf0"},"{"title":"Establish Top-notch FinTech App Security - Apriorit","url":"https://www.apriorit.com/dev-blog/create-secure-fintech-apps"},"{"title":"A Guide to Ensuring the Security of Banking and Finance Apps","url":"https://www.appknox.com/resources/guides/secure-banking-and-finance-apps"},"{"title":"Why a Privacy Budgeting App is Crucial for Secure Personal Finance Management | Pennies","url":"https://getpennies.com/privacy-budgeting-app-importance/"},"{"title":"Best Practices for Secure Data Encryption in Financial Applications | by Puneett Bhatnagr | Medium","url":"https://medium.com/@puneett.bhatnagr/best-practices-for-secure-data-encryption-in-financial-applications-0ef8da344be3"},"{"title":"Fintech App Security: Building a Secure Fintech App","url":"https://neontri.com/blog/fintech-app-security/"}]

Here are the key findings about personal finance app encryption strategy and when to enable it:

Encryption as Essential Protection

End-to-End Encryption (E2EE) helps ensure that sensitive data such as account numbers, personal identification information, addresses, passwords, transaction data cannot be intercepted or accessed by hackers or unauthorized parties. Companies developing fintech or financial apps are increasingly using encryption methods to protect data stored therein from security events such as data breaches.

When to Enable Encryption

It is not sufficient to only use HTTPS or SSL for data transport; all of your databases must also be encrypted. The Federal Financial Institutions Examination Council recommends that banks and other financial institutions encrypt all personal information users provide, all transaction information, and information from rendering financial services.

Implementation Strategy

Only when a user chooses to sync will data be shared, further safeguarded by end-to-end encryption. Robust encryption, minimizing offline storage, running regular updates, and adding hardware-backed security features are recommended practices.

Complementary Security Measures

Beyond encryption, the most effective cybersecurity solutions for fintech apps include multi-factor authentication and biometric access controls to secure user accounts; end-to-end encryption and secure APIs to protect data in transit; and real-time threat detection and fraud monitoring.

The consensus from financial security experts is that encryption should be enabled by default for all sensitive financial data both at rest in databases and in transit across networks.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. assistant (2026-05-31T14:28:56.279Z)

Now let me search for more specific information on SQLCipher operational details and more tools:

29. user (2026-05-31T14:29:03.530Z)

Web search results for query: "SQLCipher WAL mode backup restore corruption"

Links: [{"title": "Data Security and Backup Strategies in SQLite: Ensuring Data Integrity and Protection", "url": "https://www.sqliteforum.com/p/data-security-and-backup-strategies-in-sqlite-ensuring-data-integrity-and-protection"}, {"title": "SQLCipher 4.14.0 Release - Updates - Zetetic Community Discussion", "url": "https://discuss.zetetic.net/t/sqlcipher-4-14-0-release/7186"}, {"title": "Database is locked after encryption - SQLCipher - Zetetic Community Discussion", "url": "https://discuss.zetetic.net/t/database-is-locked-after-encryption/3679"}, {"title": "Power failure causes corruption in SQLite file · Issue #1482 · couchbase/couchbase-lite-ios", "url": "https://github.com/couchbase/couchbase-lite-ios/issues/1482"}, {"title": "SQLite User Forum: How exactly does corruption happen during WAL checkpoint?", "url": "https://sqlite.org/forum/info/47107ab818977549?t=h"}, {"title": "Possible 3.4.0 bug encrypting a database in WAL journal_mode - SQLCipher - Zetetic Community Discussion", "url": "https://discuss.zetetic.net/t/possible-3-4-0-bug-encrypting-a-database-in-wal-journal-mode/1324"}, {"title": "SQLite User Forum: Hot backup database in WAL mode by coping", "url": "https://sqlite.org/forum/forumpost/2ea989bbe9"}, {"title": "Failure to encrypt SQLite database when using WAL journaling · Issue #1233 · couchbase/couchbase-lite-ios", "url": "https://github.com/couchbase/couchbase-lite-ios/issues/1233"}, {"title": "How to Implement Encryption with SQLCipher", "url": "https://oneuptime.com/blog/post/2026-02-02-sqlcipher-encryption/view"}, {"title": "SQLite Corruption with fs.copyFile() in WAL Mode - Scott Spence", "url": "https://scottspence.com/posts/sqlite-corruption-fs-copyfile-issue"}]

Based on the search results, here are the key findings about SQLCipher, WAL mode, backup, restore, and corruption:

Critical WAL Mode Corruption Issue

SQLCipher 4.14.0 includes a critical upstream fix from SQLite 3.51.3, related to a WAL-reset database corruption bug, and applications running SQLCipher in WAL mode are strongly advised to upgrade.

Backup and Restore Best Practices

****WAL Mode Backup Challenges:**** When using simple file copying methods like `fs.copyFile()`, you're only copying the `.db` file while the `.db-wal` and `.db-shm` files are still being written to, which creates an inconsistent snapshot and causes instant corruption.

****Proper Restoration:**** When restoring from backup, you should remove WAL and shared memory files after restoring the database file to ensure database integrity.

****Before Backup:**** Write-Ahead Logging (WAL) mode improves durability and reduces the risk of corruption, with WAL mode ensuring that transactions are preserved even if a crash occurs.

Encryption with WAL Issues

When encryption failures occur in WAL mode, the `.wal` file may not be encrypted while the `.shm` and `.sqlite` files are encrypted, and if you delete the `.wal` and `.shm` files, you can open the database file correctly.

Consistent Backup Approach

You will have to take and hold an "intent to write" (`BEGIN IMMEDIATE`) to prohibit any other connection from writing to the wal file.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. user (2026-05-31T14:29:03.728Z)

Web search results for query: "SQLite3MC encryption library Python"

Links: [{"title": "GitHub - utelle/apsw-sqlite3mc: Another Python SQLite wrapper bundled with SQLite3 Multiple Ciphers · GitHub", "url": "https://github.com/utelle/apsw-sqlite3mc"}, {"title": "apsw-sqlite3mc · PyPI", "url": "https://pypi.org/project/apsw-sqlite3mc/"}, {"title": "apsw-sqlite3mc 3.50.4.0 released", "url": "https://groups.google.com/g/python-sqlite/c/kopDU5LR5oY"}, {"title": "Working with crypted SQLite database!", "url": "https://groups.google.com/g/jam-py/c/bnTIpJBZ1og"}, {"title": "charles leifer | Encrypted SQLite Databases with Python and SQLCipher", "url": "https://charlesleifer.com/blog/encrypted-sqlite-databases-with-python-and-sqlcipher/"}, {"title": "GitHub - coleifer/cysqlite: python sqlite driver · GitHub", "url": "https://github.com/coleifer/cysqlite"}, {"title": "Securing Your Data: How to Encrypt SQLite Databases in Python | IT trip", "url": "https://en.ittrip.xyz/python/encrypt-sqlite-python"}, {"title": "Database encryption - Python Connector for SQLite", "url": "https://docs.devart.com/python/sqlite/database-encryption.htm"}, {"title": "How to Implement Encryption with SQLCipher", "url": "https://oneuptime.com/blog/post/2026-02-02-sqlcipher-encryption/view"}]

Based on the search results, here's what I found about SQLite3MC encryption library for Python:

Overview

SQLite3 Multiple Ciphers combined with APSW (Another Python SQLite Wrapper) allows reading and writing encrypted databases in Python3 applications. The main package for this is called ****apsw-sqlite3mc****.

Installation

You can install it via pip with the command: ``pip install apsw-sqlite3mc``. Binaries are included for most platforms, and pip will build from source for the rest.

Key Features

- SQLite3 Multiple Ciphers extends SQLite 3 to allow reading and writing encrypted databases, and the distribution is entirely self contained, not altering any existing SQLite on your system
- Currently 7 different encryption cipher schemes are supported

Basic Usage

For encrypted databases you use pragmas to set a passphrase based key: ``connection.pragma("key", "my secret passphrase")`` or a binary bytes based key: ``connection.pragma("hexkey", b"\xfe\x23\x9e\x77".hex())``

If the database already has content and you want to encrypt it, use `rekey` which will modify the database to apply the supplied key.

The package is available from PyPI and has development status as production/stable as of May 6, 2026.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. user (2026-05-31T14:29:03.777Z)

Web search results for query: "pysqlcipher3 apsw pycryptodome Python SQLite encryption"

Links: [{"title": "GitHub - utelle/apsw-sqlite3mc: Another Python SQLite wrapper bundled with SQLite3 Multiple Ciphers · GitHub", "url": "https://github.com/utelle/apsw-sqlite3mc"}, {"title": "GitHub - rigglemania/pysqlcipher3: Python 3 bindings for SQLCipher · GitHub", "url": "https://github.com/rigglemania/pysqlcipher3"}, {"title": "pysqlcipher · PyPI", "url": "https://pypi.org/project/pysqlcipher/"}, {"title": "Securing Your Data: How to Encrypt SQLite Databases in Python | IT trip", "url": "https://en.ittrip.xyz/python/encrypt-sqlite-python"}, {"title": "Encrypting SQLite Database in python using pysqlitecipher module | by harsh gupta | Medium", "url": "https://medium.com/@harshnative/encrypting-sqlite-database-in-python-using-pysqlitecipher-module-23b80129fda0"}, {"title": "GitHub - privacee/pysqlcipher3: Python 3 bindings for SQLCipher · GitHub", "url": "https://github.com/privacee/pysqlcipher3"}, {"title": "How to Implement Encryption with SQLCipher", "url": "https://oneuptime.com/blog/post/2026-02-02-sqlcipher-encryption/view"}, {"title": "pysqlcipher3 · PyPI", "url": "https://pypi.org/project/pysqlcipher3/"}, {"title": "charles leifer | Encrypted SQLite Databases with Python and SQLCipher", "url": "https://charlesleifer.com/blog/encrypted-sqlite-databases-with-python-and-sqlcipher/"}]

I've found relevant information about Python SQLite encryption using `pysqlcipher3`, `APSW`, and related tools. Here's what the search results reveal:

SQLite Encryption Options in Python

pysqlcipher3

`pysqlcipher3` is an interface to the SQLite 3.x embedded relational database engine that is almost fully compliant with the Python database API version 2.0 and exposes the unique features of SQLCipher. For SQLCipher with Python, you may need to install the ``pysqlcipher3`` library using `pip`.

APSW with SQLite3 Multiple Ciphers

An alternative approach is `APSW-sqlite3mc` which extends SQLite 3 to allow reading and writing encrypted databases. For encrypted databases you need to use the relevant pragmas to set a passphrase based key, or a binary bytes based key through `connection.pragma()` calls.

Encryption Methods

SQLCipher solves encryption problems by providing transparent, full-database encryption using AES-256. When you create a new database or open an existing one, ensure you set the encryption key immediately using the SQL ``PRAGMA key`` statement.

****Note:**** The search results don't specifically mention `pycryptodome` integration with these

libraries. If you need information about using pycryptodome specifically with SQLite encryption, that may require additional research or alternative approaches to application-level encryption.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

32. user (2026-05-31T14:29:03.871Z)

Web search results for query: "Maybe app personal finance encryption"

Links: [{"title": "GitHub - maybe-finance/maybe: The personal finance app for everyone · GitHub", "url": "https://github.com/maybe-finance/maybe"}, {"title": "Maybe App Security · maybe-finance/maybe Wiki · GitHub", "url": "https://github.com/maybe-finance/maybe/wiki/Maybe-App-Security"}, {"title": "Maybe | Umbrel App Store", "url": "https://apps.umbrel.com/app/maybe"}, {"title": "Self-Hosting Maybe Finance: Open Source Personal Finance Tracker | Self Host Setup", "url": "https://selfhostsetup.com/posts/maybe-finance-personal-finance-tracker/"}, {"title": "Maybe: Personal Finance | PIGSTY", "url": "https://pigsty.io/docs/app/maybe/"}, {"title": "Maybe: Know your business, in plain English.", "url": "https://maybefinance.com/"}, {"title": "? Exploring Maybe Finance: Self-Hosting an Open Finance OS in the Cloud | by SHIV KUMAR | Medium", "url": "https://medium.com/@shivkumar0757/exploring-maybe-finance-self-hosting-an-open-finance-os-with-cloud-setup-3fb0d599be66"}, {"title": "Sure: The personal finance app for everyone by everyone", "url": "https://sure.am/"}, {"title": "Managed Maybe as a Service | Elestio", "url": "https://elest.io/open-source/maybe"}, {"title": "Show HN: Maybe ? The personal finance app for everyone | Hacker News", "url": "https://news.ycombinator.com/item?id=43865479"}]

Based on the search results, here's what I found about Maybe app and its encryption:

Overview of Maybe App

Maybe is an open-source personal finance manager. It's a fully open-source personal finance app you run on your own hardware.

Encryption Implementation

For session encryption, Maybe requires a SECRET_KEY_BASE for session encryption. This is a Rails-specific encryption key used for protecting sessions and cookies. The configuration requires users to generate a secure secret key when setting up the application.

Security Architecture

For the managed version, Maybe can be run in "self hosted" mode or "managed" mode, where the company operates the servers and database and provide a SaaS product for users, or the user fully controls the server and database. The company hosts servers and database on Render, and uses Cloudflare to mitigate network related security threats.

Current Project Status

It's worth noting that Maybe is pivoting to B2B financial forecasting and scenario planning and will no longer be actively maintaining the project. However, the app is feature-complete and actively forked by the community.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

33. assistant (2026-05-31T14:29:06.068Z)

Excellent. Let me get more specific information on licensing and a few more operational details:

34. user (2026-05-31T14:29:11.329Z)

Web search results for query: "SQLite3MC license open source"

Links: [{"title": "SQLite Copyright", "url": "https://sqlite.org/copyright.html"}, {"title": "apsw-sqlite3mc · PyPI", "url": "https://pypi.org/project/apsw-sqlite3mc/"}, {"title": "sqlite3mc - Conan 2.0: C and C++ Open Source Package Manager", "url": "https://conan.io/center/recipes/sqlite3mc"}, {"title": "Package: mingw-w64-x86_64-sqlite3mc - MSYS2 Packages", "url": "https://packages.msys2.org/packages/mingw-w64-x86_64-sqlite3mc"}, {"title": "apsw-sqlite3mc 3.51.3.1 on PyPI - Libraries.io - security & maintenance data for open source software", "url": "https://libraries.io/pypi/apsw-sqlite3mc"}, {"title": "Other FOSS License", "url": "https://docs.oracle.com/cd/F14820_01/doc.191/f15063/other_foss_license.htm"}, {"title": "Licensing of SQLite", "url": "https://sqlite-users.sqlite.narkive.com/F6leoCqK/licensing-of-sqlite"}, {"title": "Compatible License - Using SQLite [Book]", "url": "https://www.oreilly.com/library/view/using-sqlite/9781449394592/ch01s06.html"}, {"title": "LiteSync - SQLite Replication and Synchronization", "url": "https://litesync.io/en/licensing.html"}, {"title": "SQLCipher License - Commercial and Community License Information and Notices | Zetetic", "url": "https://www.zetetic.net/sqlcipher/license/"}]

Based on the search results, here's what I found about SQLite3MC license and open source status:

SQLite3MC is an SQLite3 encryption extension with support for multiple ciphers, and it is licensed under MIT. This makes it an open-source project.

The apsw-sqlite3mc package on PyPI lists the license as OSI Approved, indicating it meets open source standards.

SQLite3MC is available through various package managers and repositories. It can be installed via package managers like MSYS2 and is available on the Conan package manager for C and C++ projects. The project is actively maintained and available on GitHub under the repository <https://github.com/utelle/SQLite3MultipleCiphers>.

In summary, SQLite3MC is an open-source project released under the MIT license, which is a permissive open-source license that allows free use, modification, and distribution with minimal restrictions.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

35. user (2026-05-31T14:29:12.017Z)

Web search results for query: "SQLCipher license BSD open source"

Links: [{"title": "SQLCipher Community Edition - Open Source Information | Zetetic", "url": "https://www.zetetic.net/sqlcipher/community/"}, {"title": "sqlcipher/sqlcipher | DeepWiki", "url": "https://deepwiki.com/sqlcipher/sqlcipher"}, {"title": "Download SQLCipher for Windows 11, 10, 7, 8/8.1 (64 bit/32 bit)", "url": "https://softradar.com/sqlcipher/"}, {"title": "SQLCipher: Encrypted Database | Guardian Project", "url": "https://guardianproject.info/code/sqlcipher/"}, {"title": "Applicenses Archive - uMotif", "url": "https://umotif.com/app-licenses/trialmotif-app-licenses/"}, {"title": "SQLCipher · GitHub", "url": "https://github.com/sqlcipher"}, {"title": "SQLCipher and Licensing | B4X Programming Forum", "url": "https://www.b4x.com/android/forum/threads/sqlcipher-and-licensing.50676/"}, {"title": "sqlcipher/SQLITE_LICENSE.md at master · sqlcipher/sqlcipher", "url": "https://github.com/sqlcipher/sqlcipher/blob/master/SQLITE_LICENSE.md"}, {"title": "GitHub - sqlcipher/sqlcipher: SQLCipher is a standalone fork of SQLite that adds 256 bit AES encryption of database files and other security features. · GitHub", "url": "https://github.com/sqlcipher/sqlcipher"}]

Based on the search results, here's what I found about SQLCipher's licensing:

SQLCipher License Summary

SQLCipher Community Edition is open source under a BSD-style license. More specifically, the

project is developed by Zetetic LLC and distributed under a BSD 3-Clause license.

Usage Requirements

SQLCipher Community Edition can be used in both open source and closed source commercial software, but you must include the BSD-style license and copyright statements prominently in the application and documentation.

The most important requirement is that you must reproduce the copyright notice, license conditions, and disclaimer in your application, and you must include the full copyright notice "Copyright (c) 2008-2024, ZETETIC, LLC" and the complete license text from the SQLCipher Community Edition License in a user-accessible location.

Contribution Requirements

A current contributor license agreement is required before any patches will be looked at for consideration.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

36. user (2026-05-31T14:29:13.590Z)

Web search results for query: "keyring Python OS keychain password storage"

Links: [{"title": "Storing passwords with Python keyring - GeeksforGeeks", "url": "https://www.geeksforgeeks.org/python/storing-passwords-with-python-keyring/"}, {"title": "keyring · PyPI", "url": "https://pypi.org/project/keyring/"}, {"title": "Use keyring to store your credentials ? alexwlchan", "url": "https://alexwlchan.net/2016/you-should-use-keyring/"}, {"title": "Securely Storing Credentials in Python with Keyring | by Ryan@ | Medium", "url": "https://medium.com/@forsytheryan/securely-storing-credentials-in-python-with-keyring-d8972c3bd25f"}, {"title": "Storing and Accessing Passwords in Python Using Keyring (Windows 10 and Linux) | by Jamie Southchurch | Medium", "url": "https://jamie-southchurch.medium.com/storing-and-accessing-passwords-in-python-using-keyring-windows-10-and-linux-648cf07beef6"}, {"title": "Password Management in Python ? Keyring and Credential Manager", "url": "https://medium.com/@aarhar/password-management-in-python-keyring-and-credential-manager-29fa4ccc919e"}, {"title": "python-keyring-lib/README.rst at master · philipn/python-keyring-lib", "url": "https://github.com/philipn/python-keyring-lib/blob/master/README.rst"}, {"title": "Securely Storing Credentials in Python with Keyring - AllScient", "url": "https://www.allscient.com/post/securely-storing-credentials-in-python-with-keyring/"}, {"title": "py-keyring | MacPorts", "url": "https://ports.macports.org/port/py-keyring/details/"}]

I've performed the web search for "keyring Python OS keychain password storage". Here are the key findings:

Overview

The Python keyring library provides an easy way to access the system keyring service from python. It can be used in any application that needs safe password storage.

How It Works

The keyring module provides a wrapper around your system's password store: for example, the OS X Keychain, or the Windows Credential Vault. More specifically, it interfaces with the operating system's credential storage mechanisms, such as Keychain on macOS, Windows Credential Locker on Windows, and GNOME on Linux.

Basic Usage

The basic usage of keyring is pretty simple: just call `keyring.set_password` and

```
keyring.get_password. For example:
```python
import keyring
keyring.set_password("system", "username", "password")
keyring.get_password("system", "username")
```
```

Security Benefits

These provide much more security than keeping the password in source control, or in another file that gets included at runtime, and they're entirely decoupled from your source code. Using keyring for credential storage in Python provides a more secure and manageable approach compared to .env files and environment variables, and by leveraging the operating system's native secure storage, keyring minimizes the risk of credential exposure.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

37. user (2026-05-31T14:29:14.657Z)

Web search results for query: "file disk encryption FDE vs database encryption trade-offs"

Links: [{"title": "What is FDE Security? Full Disk Encryption Guide | Huntress", "url": "https://www.huntress.com/cybersecurity-101/topic/full-disk-encryption-guide"}, {"title": "File vs. Disk Encryption: Which Is Best for You?", "url": "https://axcrypt.net/blog/the-ultimate-guide-to-file-encryption-vs-disk-encryption-which-one-is-best-for-you/"}, {"title": "Disk vs file encryption - which is better at Data breach prevention?", "url": "https://www.secureage.com/blog/file-based-encryption-vs-full-disk-encryption"}, {"title": "File-based encryption vs full-disk encryption", "url": "https://www.hexnode.com/blogs/file-based-encryption-vs-full-disk-encryption/"}, {"title": "Disk, File and Database Encryption | Springer Nature Link", "url": "https://link.springer.com/chapter/10.1007/978-3-031-33386-6_33"}, {"title": "Disk Encryption vs. File Encryption: What's the Difference?", "url": "https://www.salvationdata.com/knowledge/disk-encryption/"}, {"title": "The Fallacy of Storage Encryption (TDE, FDE, and SSE)", "url": "https://dev.ubiqsecurity.com/docs/the-fallacy-of-storage-encryption-tde-fde-and-sse"}, {"title": "Full Disk Encryption vs File Based Encryption - The Security Buddy", "url": "https://www.thesecuritybuddy.com/encryption/full-disk-encryption-and-filesystem-level-encryption/"}, {"title": "Disk vs File Encryption: Which Is Best for You? | eSecurity Planet", "url": "https://www.esecurityplanet.com/threats/disk-vs-file-encryption-which-is-best-for-you/"}]

Based on the search results, here's an overview of the key trade-offs between Full Disk Encryption (FDE) and database encryption:

Protection Scope

Database encryption focuses specifically on protecting structured data within database systems, while it provides application-level controls that FDE cannot offer. In a database context, FDE encrypts everything on the disk (including database files, swaps, etc.), which is good for physical security, but it does not prevent an authenticated user or admin from reading any data once the system is running.

Data at Rest vs. Data in Use

FDE is useful for protecting data that is at rest, which means FDE can protect data that is stored in a device but cannot protect data in motion or use. When your system is running and you are actually logged in and using the device, or when it's moved or copied outside its security silo, data is not protected by encryption at all.

Operational Differences

While both TDE and OS-level FDE aim to protect data at rest, they operate at different layers.

TDE encrypts database files at the database engine level and can support column-level encryption and key rotation.

Practical Considerations

FDE provides broader protection but may be overkill for scenarios where only specific files contain sensitive data. For comprehensive security, you may have a false sense of security if exclusively using full disk encryption to protect your data, so it's important to consider adding a file-based approach to your current encryption practice.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

38. assistant (2026-05-31T14:29:16.555Z)

Perfect. Now let me get information on PyPI wheel availability for the key encryption libraries:

39. user (2026-05-31T14:29:22.683Z)

Web search results for query: "apsw-sqlite3mc PyPI prebuilt wheels availability"

Links: [{"title":"apsw-sqlite3mc · PyPI","url":"https://pypi.org/project/apsw-sqlite3mc/"},"{"title":"Installation and customization ? APSW 3.50.4.0 documentation","url":"https://rogerbinns.github.io/apsw/install.html"},"{"title":"GitHub - utelle/apsw-sqlite3mc: Another Python SQLite wrapper bundled with SQLite3 Multiple Ciphers · GitHub","url":"https://github.com/utelle/apsw-sqlite3mc"},"{"title":"apsw-sqlite3mc 3.51.3.1 on PyPI - Libraries.io - security & maintenance data for open source software","url":"https://libraries.io/pypi/apsw-sqlite3mc"},"{"title":"apsw-wheels","url":"https://pypi.org/project/apsw-wheels/"},"{"title":"Download ? APSW 3.37.0-r1 documentation","url":"https://czb404.github.io/apsw/download.html"},"{"title":"Pypi","url":"https://pypi.org/project/apsw-sqlite3mc/3.49.0.0"},"{"title":"pysqlite3-wheels","url":"https://pypi.org/project/pysqlite3-wheels/"},"{"title":"How to install apsw on pypy?","url":"https://groups.google.com/g/python-sqlite/c/L1ZwPfyQ4E4"},"{"title":"APSW is now available on PyPI","url":"https://simonwillison.net/2022/Sep/15/apsw/"}]

Based on the search results, here's information about apsw-sqlite3mc PyPI prebuilt wheels availability:

Prebuilt Wheels Availability

Binaries are included for most platforms, and pip will build from source for the rest. The package is available from PyPi, and you can install it with `pip install apsw-sqlite3mc`.

Supported Platforms and Python Versions

The PyPI page shows comprehensive wheel coverage across multiple platforms and Python versions (3.10 through 3.14), including:

- manylinux (glibc 2.28+ and 2.31+) for ARMv7l and ARM64
- macOS (both ARM64 and x86-64)
- Windows (including ARM64)

Recent releases include wheels for various Python versions from 3.10 onwards, with the latest builds uploaded as of May 6, 2026.

Self-Contained Distribution

An important feature is that the distribution is entirely self contained, and does not use or alter any existing SQLite you may already have on your system.

REMINDER: You MUST include the sources above in your response to the user using markdown

hyperlinks.

40. user (2026-05-31T14:29:23.331Z)

Web search results for query: "pysqlcipher3 PyPI wheels Windows Linux cp311 cp312 cp313"

Links: [{"title": "sqlcipher3-wheels · PyPI", "url": "https://pypi.org/project/sqlcipher3-wheels/"}, {"title": "GitHub - rotki/pysqlcipher3: Configuration to build and publish pysqlcipher3 with wheels for rotki", "url": "https://github.com/rotki/pysqlcipher3"}, {"title": "pysqlcipher3 · PyPI", "url": "https://pypi.org/project/pysqlcipher3/"}, {"title": "pysqlcipher3/README.md at main · rotki/pysqlcipher3", "url": "https://github.com/rotki/pysqlcipher3/blob/main/README.md"}, {"title": "piwheels - pysqlcipher3", "url": "https://www.piwheels.org/project/pysqlcipher3/"}, {"title": "Wheelodex ? rotki-pysqlcipher3", "url": "https://www.wheelodex.org/projects/rotki-pysqlcipher3/"}, {"title": "sqlcipher3 · PyPI", "url": "https://pypi.org/project/sqlcipher3/"}, {"title": "piwheels - rotki-pysqlcipher3", "url": "https://www.piwheels.org/project/rotki-pysqlcipher3/"}, {"title": "rotki-pysqlcipher3 · PyPI", "url": "https://pypi.org/project/rotki-pysqlcipher3/"}]

Based on the search results, here's what I found regarding pysqlcipher3 wheels for Windows and Linux with Python versions cp311, cp312, and cp313:

Official pysqlcipher3 Package

The pysqlcipher3 package is an interface to the SQLite 3.x embedded relational database engine that is almost fully compliant with the Python database API version 2.0. However, the official pysqlcipher3 PyPI page (pypi.org/project/pysqlcipher3/) only provides source distributions, not pre-built wheels.

Alternative: rotki-pysqlcipher3

For pre-built wheels, there's an alternative package called **rotki-pysqlcipher3** available on PyPI. This package provides wheels for CPython 3.11 and 3.12 for Windows (AMD64), macOS, and Linux (manylinux). However, this does not appear to include cp313 (Python 3.13) support based on the available information.

Related: sqlcipher3-wheels

There's also a **sqlcipher3-wheels** package on PyPI that provides pre-built wheels. This package offers wheels for cp311, cp312, and cp313 across multiple platforms including Windows and Linux variants.

The key difference is that the original pysqlcipher3 package requires libsqlcipher to be pre-installed on your system, while these alternative wheel distributions provide static builds with the dependencies included.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

41. assistant (2026-05-31T14:30:03.669Z)

Perfect! Now I have comprehensive information. Let me compile the final report:

Comprehensive Survey: Encryption at Rest for muneem (Local-First Personal Finance)

Based on research of real-world personal finance tools, SQLite encryption libraries, and operational practices, here is the state of art and a pragmatic recommendation for muneem.

Part A: How Existing Local-First Finance Tools Handle Encryption at Rest

Plaintext-by-default (relying on OS-level protection):

1. **beancount & ledger-cli** ? Plain-text ledger files (not SQLite). No database encryption; rely on FDE or file-system encryption. Simplicity is the feature. See plaintextaccounting.org and [Beancount.io](https://beancount.io/blog/2025/04/15/beancount-ecosystem).
2. **GnuCash** ? SQLite/XML storage, no built-in database encryption. Users expected to use OS FDE. [GnuCash comparison](https://slashdot.org/software/comparison/Firefly-III-vs-GnuCash/).
3. **Firefly III** ? Uses plaintext database by default. The docs explicitly recommend OS-level disk encryption for data-at-rest protection: "no encryption of data-at-rest is used, so users should use their operating system's disk encryption features." Field-level encryption (AES) was removed in favor of relying on FDE. See [Firefly III Security Docs](https://docs.firefly-iii.org/explanation/more-information/security/).

Client-side end-to-end encryption (on remote servers):

4. **Actual Budget** ? SQLite local file. For self-hosted or local-only use, data is unencrypted at rest in the .db file. **When syncing to a remote Actual Server**, the app supports end-to-end encryption: data is encrypted client-side before leaving the device; the server stores ciphertext and never holds keys. See [Actual Budget Settings](https://actualbudget.org/docs/settings/) and [FAQ](https://actualbudget.org/docs/faq/).
5. **Maybe (now archived)** ? SQLite storage, plaintext at rest. Relied on OS FDE. Now a community fork (project pivoted to B2B). See [Maybe App Security Wiki](https://github.com/maybe-finance/maybe/wiki/Maybe-App-Security).

Consensus: Established personal finance tools treat database encryption as **optional**, not default. Nearly all recommend **File Disk Encryption (FDE)** (Windows BitLocker, macOS FileVault, Linux LUKS) as the primary at-rest protection for local databases. Only Actual Budget offers database-level encryption, and only for **remote sync scenarios** (not local-only use).

Part B: SQLCipher vs SQLite3MC ? Operational Details & Security

Both are mature, permissively licensed SQLite extensions that add AES-256 transparent full-database encryption.

License & Availability

| Tool | License | Python Binding | Wheels (Win/Linux cp311?cp313) | Status |
|-----------|--|--|---|--|
| SQLCipher | BSD-3-Clause (Zetetic Community Edition) | [BSD-3-Clause (Zetetic Community Edition)](https://www.zetetic.net/sqlcipher/community/) | `pysqlcipher3` (source) or `sqlcipher3-wheels` / `rotki-pysqlcipher3` | sqlcipher3-wheels: YES; rotki-pysqlcipher3: cp311?cp312 only Mature, widely used |
| SQLite3MC | MIT | [MIT](https://github.com/utelle/SQLite3MultipleCiphers) | `apsw-sqlite3mc` (APSW binding) | [apsw-sqlite3mc: YES (cp311?cp314)](https://pypi.org/project/apsw-sqlite3mc/) Actively maintained, newer |

Verdict on licensing: Both are permissive and safe for commercial use. SQLCipher is BSD (community edition); SQLite3MC is MIT. Both are acceptable per muneem's DEPENDENCIES.md rules.

Verdict on wheels: **apsw-sqlite3mc wins here.** It has prebuilt wheels for cp311?cp313 (and cp314) across Windows + Linux + macOS, maintained actively, entirely self-contained. sqlcipher3-wheels also covers cp313, but apsw-sqlite3mc is the actively maintained option. pysqlcipher3 bindings are less reliable for prebuilt wheels.

Operational Issues

WAL Mode & Backup/Restore:

SQLCipher (and SQLite3MC) both suffer the same WAL (Write-Ahead Logging) complications:

- **Backup Problem:** Simple file copy (``shutil.copy()`, `cp`, `fs.copyFile()``) of the `.db`` file while WAL is active leaves `.db-wal`` and `.db-shm`` files behind, creating an **inconsistent snapshot ? instant corruption** on restore.
- **Safe Backup:** Must use ``BEGIN IMMEDIATE; backup_api()`` or ``VACUUM;`` before copying, OR delete `.wal`` + `.shm`` files after restore.
- **Critical Fix:** SQLCipher 4.14.0+ includes a WAL-reset database corruption fix from SQLite 3.51.3. Apps in WAL mode must upgrade. See [SQLCipher 4.14.0 Release](https://discuss.zetetic.net/t/sqlcipher-4-14-0-release/7186).
- **WAL Encryption Bug:** In older versions, `.wal`` file could remain unencrypted while `.db`` and `.shm`` are encrypted; deleting `.wal`` + `.shm`` after backup restore works around this. See [Couchbase Lite Issue #1233](https://github.com/couchbase/couchbase-lite-ios/issues/1233).

Query Performance:

Both tools add overhead:

- Encryption/decryption per page (4 KB by default)
- Negligible on typical personal finance DB sizes (< 100 MB, < 1M rows)
- Noticeable on very large tables (> 10M rows), but muneem is single-user, year-scale data

Migration & Debugging:

- **Cannot open encrypted DB with plain `sqlite3` CLI.** Must use ``sqlcipher`` binary or Python binding.
- **Key rotation:** SQLCipher/SQLite3MC support ``PRAGMA rekey`` to change the encryption key.
- **Transparent queries:** Both preserve full SQL queryability (WHERE, ORDER BY, aggregates) on encrypted data ? the key insight for muneem.

Security Advisories

SQLCipher CVEs:

- [CVE-2020-27207](https://www.cvedetails.com/cve/CVE-2020-27207/): Use-after-free in SQLCipher 4.x < 4.4.1 (via `sqlcipher_codec_pragma`). **Fixed in 4.4.1+.**
- [Critical DoS in older versions](https://security.telekom.com/): SQL injection via crafted queries to trigger DoS. **Fixed in 4.5.3+.**
- **No known critical CVEs in recent versions (4.5+).**

SQLite3MC:

- No dedicated CVE database, but inherits SQLite's CVE fixes. Actively maintained; latest stable is 3.51.3 (bundled in `apsw-sqlite3mc 3.51.3.1`).

Part C: Pragmatic Phased Recommendation for muneem

Given that muneem is **just starting to store real numbers** and the design constraints are 100% local + transparent SQL queryability, here is the recommended approach:

Phase 1 (NOW ? Foundation: FDE + Keychain-cached decryption key)

Adopt:

- **File Disk Encryption (FDE) for the DB file.** Recommend users enable OS-level encryption (Windows BitLocker, macOS FileVault, Linux LUKS). This is zero-cost, widely understood, and aligns with all existing personal finance tools.
- **OS Keychain for the (future) database encryption key.** muneem already uses ``keyring`` (MIT) for Gmail/Outlook OAuth secrets. Extend it to store a database encryption key once encryption is added. Keys are managed by Windows Credential Manager / macOS Keychain / GNOME Keyring ? never written to disk in plaintext.

Why this is sufficient now:

1. FDE provides solid at-rest protection for a development-phase tool.
2. Operational simplicity: no backup/restore complications, no WAL edge cases, no tool incompatibilities.
3. Preserves the option to add transparent database encryption later without refactoring the DB layer.
4. Aligns with how Firefly III, GnuCash, and beancount handle local DBs.

****Code impact:**** Zero changes to db.py. Merely document the FDE recommendation in DESIGN.md § Security (rule 4).

****Phase 2 (TRIGGER: When storing real multi-year balances, or before public alpha release)****

****Adopt: apsw-sqlite3mc (or pysqlcipher3 if you prefer SQLCipher).****

****Why apsw-sqlite3mc is the choice:****

1. ****Wheel availability:**** cp311?cp313 prebuilt wheels for Windows, Linux, macOS. Zero compilation friction.
2. ****License:**** MIT, permissive, commercially safe.
3. ****Actively maintained:**** Latest release is 3.51.3.1 (May 2026), tracks SQLite security patches closely.
4. ****Clean Python API:**** `connection.pragma("key", password)` ? tiny refactor to db.py.
5. ****Transparent queries:**** No impact on CLI query logic.

****Migration path:****

```python

## db.py ? sketch of the change

```
def connect(db_path: Path | str, encryption_key: str | None = None) ->
Iterator[sqlite3.Connection]:
 """Open a connection, optionally encrypted."""
 path = str(db_path)
 if path != ":memory:":
 Path(path).parent.mkdir(parents=True, exist_ok=True)

 import apsw_sqlite3mc as sqlite3 # Use encrypted binding instead of stdlib sqlite3
 conn = sqlite3.connect(path)

 if encryption_key:
 conn.pragma("key", encryption_key) # Set encryption key (or PRAGMA rekey for existing
DB)

 conn.row_factory = sqlite3.Row
 conn.execute("PRAGMA foreign_keys = ON")
 # ... rest unchanged
...
```

**\*\*Key procurement:\*\*** Fetch from OS keychain via `keyring` module (already a dependency). On first run, generate a random key, store it in keychain, use it for all future connections.

**\*\*Backup/restore safety:\*\***

- Document the safe backup procedure: `VACUUM;` before copying, or delete `.db-wal` + `.db-shm` after restore.
- OR use SQLite's backup API (available in apsw\_sqlite3mc).

**\*\*When to trigger Phase 2:\*\***

- [ ] User requests export of real financial data to a different machine.
- [ ] Before shipping a version that stores real account numbers / balances.
- [ ] If a user reports storing > 3 years of real data and wants stronger protection.

#### **\*\*Phase 2b (Optional upgrade within Phase 2): Full database-level + FDE redundancy\*\***

Once Phase 2 is in place, FDE + database encryption is defense-in-depth:

- **\*\*FDE\*\*** protects against stolen hardware.
- **\*\*Database encryption\*\*** protects against extracted files (e.g., from a backup, or if FDE is somehow bypassed).
- Both are transparent to the CLI and queries.

---

## Part D: Summary Table & Recommendation

| Aspect              | Phase 1 (NOW)       | Phase 2 (Later)                                          |
|---------------------|---------------------|----------------------------------------------------------|
| Database encryption | None (rely on FDE)  | apsw-sqlite3mc (MIT, wheels available)                   |
| Key storage         | N/A                 | OS keychain (via `keyring` lib, already used)            |
| Transparent queries | Yes                 | Yes (full SQL unchanged)                                 |
| Backup complexity   | Low                 | Medium (VACUUM before copy, or delete WAL after restore) |
| Operational burden  | Zero                | Low (one-time setup, then transparent)                   |
| Cost                | Free                | Free (open source)                                       |
| Commercial safety   | Safe (no licensing) | Safe (MIT license)                                       |

#### #### \*\*Recommended Action Now\*\*

1. **Phase 1 is not a code change ? it's a documentation update:**
  - Add to DESIGN.md § Security, rule 4: **"DB encrypted at rest: Recommend users enable OS FDE (BitLocker/FileVault/LUKS). Database-level encryption via SQLCipher/SQLite3MC is deferred to Phase 2."**
  - Update README.md / docs/ingesting-statements.md: **"For production use with real data, enable full-disk encryption on your system."**
2. **Leave db.py unchanged** for now. The stdlib `sqlite3` module is fine until Phase 2.
3. **Open a ROADMAP task (e.g., D.2)** tracking: **"Implement transparent database encryption (Phase 2) ? trigger: before storing multi-year real data, or on user request. Use apsw-sqlite3mc + OS keychain."**
4. **Test the Phase 2 migration** in a branch (optional now): Verify that switching from stdlib `sqlite3` to `apsw\_sqlite3mc` is a drop-in replacement and that existing queries work unchanged.

---

#### Sources Cited

- [Actual Budget ? Local-first sync & E2EE](<https://actualbudget.org/docs/settings/>)
- [Firefly III Security Docs ? No DB encryption, recommend FDE](<https://docs.firefly-iii.org/explanation/more-information/security/>)
- [Beancount & Plain Text Accounting](<https://beancount.io/blog/2025/04/15/beancount-ecosystem>)
- [SQLCipher Community Edition License](<https://www.zetetic.net/sqlcipher/community/>)
- [SQLCipher 4.14.0 Release ? WAL corruption fix](<https://discuss.zetetic.net/t/sqlcipher-4-14-0-release/7186>)
- [apsw-sqlite3mc on PyPI](<https://pypi.org/project/apsw-sqlite3mc/>)
- [SQLite3MC GitHub (MIT license)](<https://github.com/utelle/SQLite3MultipleCiphers>)
- [Python keyring docs & GeeksforGeeks tutorial](<https://pypi.org/project/keyring/>)
- [FDE vs Database Encryption trade-offs](<https://axcrypt.net/blog/the-ultimate-guide-to-file-encryption-vs-disk-encryption-which-one-is-best-for-you/>)
- [CVE-2020-27207 (SQLCipher 4.x < 4.4.1)](<https://www.cvedetails.com/cve/CVE-2020-27207/>)
- [WAL Backup/Restore Best Practices (SQLite Forum)](<https://sqlite.org/forum/forumpost/2ea989bbe9>)